Confidence-Aware Local–Remote Language Model Routing for Software Engineering

A cost–accuracy study with verified teacher distillation

Author name and university affiliation to be completed

25 October 2026 — pre-results manuscript

Contents

Manuscript status **3**

Abstract **4**

Acknowledgement **5**

1 Introduction **6**
1.1 Motivation . . . 6
1.2 The research problem . . . 6
1.3 Scope and terminology . . . 7
1.4 Research questions . . . 7
1.5 Intended contribution . . . 8
1.6 Organization . . . 8

2 Foundations and related work **10**
2.1 Review scope and evidence quality . . . 10
2.2 From code generation to software engineering . . . 11
2.3 Selective prediction and learning to defer . . . 12
2.4 Calibration, discrimination and distribution shift . . . 13
2.5 Budget-aware routing and cascades . . . 14
2.6 Speculative decoding as inspiration, with different guarantees . . . 15
2.7 Distillation and progressive local capability . . . 16
2.8 Parameter-efficient adaptation and AI feedback . . . 16
2.9 Benchmark validity, contamination and harness effects . . . 17
2.10 Synthesis and research contribution . . . 18

3 System design and formalization **20**
3.1 Design principles . . . 20
3.2 State and action spaces . . . 20
3.3 Estimating benefit rather than asking for confidence . . . 22
3.4 Sequential delegation and state continuity . . . 23
3.5 Confidence-scheduled verification as an analogy . . . 23
3.6 Student learning mechanisms . . . 23
3.7 Adaptation and routing as separate interventions . . . 24
3.8 Cost model . . . 24
3.9 Evidence record . . . 25

4 Model and price landscape **26**
4.1 Candidate selection . . . 26
4.2 Price comparison and what it measures . . . 28
4.3 Published accuracy evidence: contextual, not a leaderboard . . . 29
4.4 Runtime readiness and the first bounded study . . . 30

5 Published open- and closed-weight software-engineering accuracy 32

6 Experimental methodology 34
6.1 Research questions and confirmatory claims . . . 34
6.2 What “single LLM” means in this comparison . . . 35
6.3 Benchmark selection and target population . . . 35
6.4 Development data, test isolation and repository familiarity . . . 36
6.5 Common agent scaffold and execution controls . . . 37
6.6 Models, release freshness and experimental arms . . . 38
6.7 Router training and deployment policy . . . 39
6.8 Escalation and confidence features acquired during execution . . . 40
6.9 Outcome, router-quality and robustness measurements . . . 41
6.10 Economic accounting and pricing comparisons . . . 41
6.11 Statistical analysis and sample-size planning . . . 42
6.12 Student adaptation and logit-distillation extensions . . . 43
6.13 Resource plan, execution stages and stopping rules . . . 44
6.14 Reproducibility, artifacts and completion criteria . . . 44

7 Analysis and reporting plan 46
7.1 Estimands and success criteria . . . 46
7.2 Accuracy and cost frontiers . . . 47
7.3 Router quality as a prediction problem . . . 47
7.4 Statistical uncertainty and dependence . . . 48
7.5 Power and the role of the feasibility pilot . . . 48
7.6 Prespecified outputs . . . 49
7.7 Result-dependent discussion to be written later . . . 49

8 Reproducibility, ethics and limitations 50
8.1 Implementation state . . . 50
8.2 Reproducibility package . . . 51
8.3 Task management and publication . . . 51
8.4 Data and research ethics . . . 51
8.5 Construct validity . . . 52
8.6 Internal validity . . . 52
8.7 External validity . . . 53
8.8 Resource and completion boundaries . . . 53

Results and discussion reserved for completion 54

Appendix: empty reporting templates 55
Main system comparison . . . 55
Routing and student diagnostics . . . 55
Run manifest checklist . . . 55

References 57

Manuscript status

This document is a master's thesis manuscript in artificial intelligence, prepared before the proposed repository-level software-engineering evaluation. Its research questions, background, system design, experimental protocol, analysis plan and limitations are written in advance. **The empirical results, result-dependent discussion and empirical conclusion are intentionally not written.** No claim that the proposed dual-model system improves accuracy, reduces measured cost or benefits from distillation is made.

A historical programming-problem replay and software-validation artifacts exist in the accompanying repository. They are development material for a different task distribution, and their measurements are not presented as evidence for this thesis's software-engineering hypotheses. The prospective repository-level study requires its own frozen model catalog, evaluation records and analysis. Published third-party benchmark claims, where included in the model-landscape chapter, are clearly distinguished from the proposed study's unobserved results.

The source and pricing review has a cutoff of **25 October 2026**. A model catalog is a dated snapshot, not a promise that every listed model is deployable on the available hardware or that a floating API alias will remain unchanged. Secondary price observations require vendor confirmation before purchasing inference. The existing US$100 Runpod authorization funds a bounded feasibility pilot; it does not fund the entire proposed confirmatory matrix or additional closed-provider API spending.

Author, institution, degree wording, supervisor, institutional declarations and submission date are to be completed by the author. The manuscript does not invent those details or represent that an institution has approved the research.

Abstract

Large language models increasingly support software-engineering work through tools that inspect repositories, edit code and execute tests. High-capability models can be expensive to operate, while smaller models can run on user-owned hardware but may fail on tasks requiring substantial repository understanding or multi-step repair. A dual-model system seeks to reconcile these constraints by assigning suitable work to a local model and delegating selected work to a stronger remote model. Its value depends not merely on reducing remote calls, but on retaining useful software-engineering accuracy after accounting for routing, verification, failed attempts and local computation.

This thesis proposes a controlled evaluation of confidence-aware local–remote routing for repository-level issue resolution. The design compares a dual-model system with local-only and remote-only agents using the same tool harness, task environments and stopping rules. Random and rule-based routing, matched on estimated cost using calibration data, establish allocation baselines; a separate no-tools language-model condition examines the contribution of the surrounding agent system without conflating it with routing. The router estimates the expected quality improvement and incremental cost of delegation, with calibration and action thresholds selected on development data. A secondary study investigates whether verified teacher-output distillation improves the local model and changes the achievable quality–cost frontier.

The proposed evaluation uses pinned software-engineering benchmarks, paired task outcomes, held-out calibration, repository-level generalization checks and uncertainty estimates that respect task dependence. Financial accounting distinguishes paid API tokens from local hardware and self-hosted GPU costs. Current open-weight and closed-model candidates are assessed through a dated evidence catalog; published performance claims with different harnesses are not treated as controlled comparisons. The intended contribution is a reproducible account of when dual-model delegation is worthwhile, how reliably its confidence estimates support decisions, and which costs or failure modes limit its practical use. The manuscript reports no empirical outcomes for these questions.

Keywords: software-engineering agents; model routing; selective prediction; calibrated confidence; cost-sensitive learning; knowledge distillation; open-weight language models.

Acknowledgement

The project’s guiding sentiment is: “Believe in me who believes in you.”

Chapter 1

Introduction

1.1 Motivation

The useful output of a software-engineering assistant is a correct and reviewable change to a software system. Generating plausible code is only one step toward that output. A repository task may require identifying an unfamiliar component, reproducing a failure, reading conventions, proposing a change, running tests and responding to intermediate evidence. These activities make the model, its tools and its execution environment a combined decision-making system. An evaluation that measures isolated answer quality does not fully characterize that system.

Current high-capability language models offer a convenient route to strong assistance through hosted APIs. However, their operating cost grows with repeated context processing, long trajectories, retries and concurrent use. A small local model changes the cost structure: the user may avoid an inference-service charge for individual calls, but still bears hardware, memory, energy and latency costs. The local model may also require more attempts or fail on tasks a stronger model could complete. A meaningful comparison must therefore consider the total cost of successful work rather than token prices alone.

TTGL proposes a local-first coding assistant that combines these resources. A local open-weight model performs work when the system expects it to be sufficient. A remote model is available when delegation is likely to improve the outcome enough to justify its cost. Customers may select an eligible remote model; the product selects an appropriate local model for the device. Verified remote outputs can later become training examples for local adaptation. This architecture creates an appealing hypothesis: the system could preserve much of a stronger model's useful capability while reducing paid inference and increasing local coverage over time. The hypothesis requires an experiment. Neither a router's existence nor a small model's high confidence establishes it.

1.2 The research problem

The central question is how to allocate heterogeneous model capability to software-engineering work under cost and quality constraints. A simple "easy local, difficult remote" rule is incomplete. A task may be difficult for both models; remote execution then adds cost with little improvement. Conversely, the remote model may bring a predictable benefit on an apparently routine task where local

behavior is unreliable. Some tasks may be solved by the local model and missed by the remote one. The relevant quantity is therefore conditional **benefit of delegation**, not difficulty or local uncertainty alone.

The problem also changes with the point of intervention. A router can choose a model before any generation, after a bounded local attempt, or at successive tool boundaries during an agent trajectory. These settings have different information and costs. A router that observes a failed test has already paid for local reasoning, editing and verification; a pre-call router has not. Treating both as if they selected one equally priced answer hides the economics and confounds experimental attribution.

Training the local model creates a second problem. Improvements in apparent local coverage can arise from a better student, a recalibrated router, a changed workload, altered verification rules or easier task selection. A thesis must distinguish these explanations. It must also separate sequence distillation from token-distribution distillation, and supervised adaptation from learning with AI-generated preference feedback. These methods require different evidence and different resources.

1.3 Scope and terminology

In this manuscript, a **language model** is the generative model queried to produce text or structured actions. An **agent harness** manages the interaction between that model, repository state, tools and termination conditions. A **single-model agent** can invoke many tools and make many calls, but uses one model throughout its trajectory. A **dual-model system** uses a policy that can select between a local and a remote model. It may still use only one model on an individual task.

“Local” identifies an intended deployment on user-controlled hardware. “Remote” identifies an endpoint outside that device. An open-weight model can be served remotely, and a model with an open-source serving engine is not automatically an open-source model in the same licensing sense. The thesis uses **open-weight** when weights are available under a published license and **closed model** when the weights are unavailable to the researcher. Software licenses, weight licenses, API terms and training-data permissions are recorded separately.

The primary outcome concerns repository-level issue resolution, as assessed by a pinned execution-based benchmark. Isolated programming problems serve only as auxiliary diagnostics or development data with their own scope. The experiment does not equate passing benchmark tests with universal correctness, security, maintainability or acceptance by a human reviewer. These are separate dimensions that would need additional study.

1.4 Research questions

The thesis organizes the empirical work around five questions:

1. **System effectiveness:** Can a dual-model agent retain issue-resolution accuracy within a prespecified tolerance of a strong single-model agent while reducing total inference and execution cost?

2. **Routing quality:** Does a learned, calibrated delegation policy outperform random and simple rules at comparable spending, and does predicted benefit outperform local-failure confidence alone?
3. **Operational tradeoff:** How do the attainable accuracy–cost frontier and latency distribution change across models, workload strata, local hardware profiles and deployment modes?
4. **Student improvement:** Does verified teacher-output distillation improve held-out local capability, and how much of any subsequent system improvement is attributable to the student rather than router recalibration?
5. **Market relevance:** How do eligible current open-weight and closed-model alternatives compare under the same experimental conditions, and how do dated published price/performance observations inform—but not replace—that evaluation?

The methodology chapter defines the primary contrasts and multiplicity rules. The questions are deliberately broader than the confirmatory hypotheses: exploratory diagnostic analyses are useful, but they cannot replace a failed or inconclusive primary comparison.

1.5 Intended contribution

This thesis does not claim to invent model routing, confidence calibration or distillation. The intended contribution is a controlled evaluation of their interaction in software engineering. The study specifies a common harness, paired task evaluations, a router whose information is available at the time of decision, explicit uncertainty and calibration checks, and an accounting scheme that charges unsuccessful attempts and verification. A separate adaptation arm provides a way to test whether verified teacher data shifts the local model's frontier.

The accompanying research software already provides data validation, a historical routing-replay implementation, teacher-example preparation and a training/inference scaffold. The prospective repository-level adapter and controlled multi-model evaluation remain to be executed. Reusing established harnesses, inference runtimes and training libraries reduces engineering effort and improves comparability, while project-specific code is confined to routing policy, experiment integration and evidence recording.

The contribution is valuable whether the eventual result is positive, negative or uncertain. A dual-model system may be attractive only for particular task classes, hardware utilization or cost ratios. A simple rule may outperform a learned router under distribution shift. Distillation may improve one class of tasks while damaging another. The experiment is designed to make such outcomes observable instead of selecting only favorable cases.

1.6 Organization

The next chapter reviews the conceptual foundations and related work. The system-design chapter defines state, decisions and learning mechanisms. A dated model-landscape chapter records candidate models, pricing evidence and the limits of published benchmark comparisons. The methodology specifies the

controlled software-engineering evaluation, followed by a prespecified analysis and reporting plan. The final substantive chapter discusses reproducibility, ethics, threats to validity and the conditions required to complete the study. Results and result-dependent conclusions are reserved for later completion.

Chapter 2

Foundations and related work

2.1 Review scope and evidence quality

This chapter develops the conceptual basis for a local/remote software-engineering assistant and identifies the claims that the proposed experiment can test. Its scope is deliberately broader than model routing alone. A useful system must choose a model, obtain a usable solution through an interaction interface, verify that solution, and account for the resources consumed by both successful and unsuccessful attempts. The relevant literature therefore spans selective prediction, learning to defer, LLM routing, agent-computer interfaces, code evaluation, knowledge distillation and parameter-efficient adaptation.

The review is a focused narrative synthesis, not a systematic review or meta-analysis. Source discovery combined exact-title lookup, author-maintained repositories and backward citation following. The review cutoff and retrieval date are 25 October 2026. Exact-title lookup targets included “RouteLLM: Learning to Route LLMs with Preference Data”, “FrugalGPT”, “Consistent Estimators for Learning to Defer to an Expert”, “On Calibration of Modern Neural Networks”, “Distilling the Knowledge in a Neural Network”, “Dynamic Speculation Lookahead”, “SWE-agent” and “SWE-bench”. The bibliography records the corresponding titles, stable paper identifiers and software revisions, allowing another reviewer to reconstruct the selected corpus. No broad scholarly-index search or database hit-count screening was conducted.

Inclusion required a source to define a relevant method, benchmark or implementation, with identifiable authorship and primary provenance. Foundational papers were retained even when their experimental domain preceded LLMs; contemporary papers and software were included when they changed the available routing, training or evaluation choices. Exclusions covered untraceable claims, advertising or leaderboard rank without evaluation context, duplicate descriptions of the same artifact, and material unrelated to the study’s decision or validity questions. These rules bound a purposive review rather than guarantee exhaustive coverage. The research gap is consequently a motivated empirical question, not a claim that no related system has ever been studied.

Evidence quality varies across sources. Relevant definitions, methods and discussion were inspected in primary full texts for sixteen works: calibration, SelectiveNet, learning to defer, FrugalGPT, RouteLLM, speculative decoding, dynamic speculation, distillation, LoRA, QLoRA, DPO, Constitutional AI, DeepSeek-

R1, SWE-agent, SWE-bench and SWE-bench Pro. This is targeted section-level reading, not a claim that every proof, appendix and empirical table was independently audited. DeepInfer's confidence head and scheduling code were inspected at a pinned revision, and its claims remain implementation evidence. Other benchmark and SDK references rely on official repository documentation; their bibliographic notes identify this narrower evidence. The review does not treat authors' reported improvements as a replication or as predicted thesis outcomes.

Fast-moving software requires a second form of provenance. A paper's method, a repository's current code and a hosted product's current behavior need not coincide. This chapter separates paper citations from versioned software-documentation citations. SWE-bench, mini-SWE-agent and SWE-bench Pro documentation were inspected on 25 October 2026; their relevant revisions are recorded in the bibliography. Descriptions below establish the documented scope of these systems, not independent performance measurements. This pre-results manuscript makes no claim about the proposed system's measured accuracy, cost or training benefit.

2.2 From code generation to software engineering

A code-generation task usually supplies a specification and asks for a function or program that satisfies a collection of tests. HumanEval made executable functional correctness a practical evaluation target, while subsequent work such as EvalPlus strengthened the tests used to assess generated functions (M. Chen et al. 2021; Liu et al. 2023). These settings isolate an important capability, but successful synthesis of a self-contained function is only one component of software development.

A repository-level software-engineering task supplies an issue and a codebase at a particular revision. The system must identify relevant files, understand interfaces, modify implementation code, preserve existing behavior and produce a patch that survives evaluation. SWE-bench operationalizes this setting through real repository issues and corresponding evaluation machinery (Jimenez et al. 2024). BigCodeBench occupies a complementary position by emphasizing diverse library calls and complex instructions within function-level problems (Zhuo et al. 2024). LiveCodeBench provides temporally organized programming tasks useful for contamination-aware coding evaluation (Jain et al. 2024). None of these benchmark categories should be substituted for another without narrowing the claim accordingly.

An LLM and an agent are also distinct experimental objects. An LLM maps context to a distribution over outputs. An agent couples that model to tools, state, stopping rules, retrieval, execution and feedback. A model's repository-task performance is therefore conditional on an agent-computer interface and its resource limits. SWE-agent directly studies that interface, and its current maintainers recommend the simpler mini-SWE-agent as the successor for ongoing use (Yang et al. 2024; SWE-agent Team 2026). OpenHands supplies a further example of separating agents, tools, conversations, workspaces and server interfaces within a reusable SDK (Wang et al. 2025). These systems support the methodological choice to reuse a maintained harness rather than invent a new one merely to demonstrate routing.

The appropriate primary comparison is thus a dual-model agent against single-

model agents using the same harness, repository snapshot, available tools and evaluation rules. Comparing a tool-using dual system against a single bare prompt would confound routing with access to feedback and execution. A no-tools LLM can remain an informative secondary ablation, provided it is labelled as a different interface condition. Likewise, a benchmark score obtained with one vendor's proprietary agent should not be treated as the score of its underlying model under the thesis harness.

2.3 Selective prediction and learning to defer

Selective prediction studies a predictor that may abstain rather than produce an answer for every input. Its central quantities are coverage, the fraction of inputs accepted, and selective risk, the error rate among accepted inputs. SelectiveNet integrates prediction and rejection within a learned system (Geifman and El-Yaniv 2019). This perspective helps distinguish a useful local model from a merely overconfident one: increasing local coverage is desirable only if the accepted local work maintains adequate quality.

An assistant with a remote model is more accurately viewed as a learning-to-defer system because abstention invokes another decision-maker. Learning-to-defer formulations explicitly model the expert alternative rather than assume that rejection is costless (Mozannar and Sontag 2020). Here the remote model is an imperfect expert. It may solve a problem that defeats the local model, fail alongside it, or occasionally damage a solution the local model would have produced correctly. The experimental target must represent all of these cases.

Let X denote information available when a routing decision is made, and let $Y_s, Y_l \in \{0,1\}$ denote acceptance of a solution generated by the small and large systems under specified protocols. The local-success probability is $p_s(x) = P(Y_s = 1 \mid X = x)$. A policy that delegates whenever $p_s(x)$ is low assumes that local failure predicts useful remote intervention. A more directly aligned quantity is

$$g(x) = \mathbb{E}[Y_l - Y_s \mid X = x].$$

This expected benefit can be positive, zero or negative. It differs from local uncertainty because a difficult problem can defeat both systems. When cost varies across tasks, delegation can instead depend on expected benefit relative to incremental cost, with explicit constraints on latency, budget and permitted destinations. This formulation is a decision rule to be evaluated; it does not establish that a cheaply estimated $g(x)$ will generalize.

Paired model outcomes on the same task provide useful training information, but their interpretation depends on the execution design. If models receive different tools, feedback or stopping budgets, the outcome difference describes different systems rather than model substitution alone. Repeated stochastic samples provide an estimate of each system's task success distribution, not interchangeable independent tasks. Model sampling seeds also need not pair meaningfully across different providers. The experiment should preserve task-level pairing while recording the conditions that generated each outcome.

Operational logging introduces selective labels: a deployed router observes the

remote outcome mainly on tasks it already chose to escalate. Training solely on those logs can reinforce its original selection bias. Controlled paired data, offline shadow evaluations or carefully budgeted exploration can provide information about counterfactual choices. Such exploration must be distinguished from extra user-visible attempts, and its cost belongs in the system's accounting.

2.4 Calibration, discrimination and distribution shift

Confidence is useful only after its target has been defined. Next-token probability expresses the model's distribution over a token; it is not automatically the probability that a completed patch resolves a repository issue. Natural-language statements of confidence introduce additional sensitivity to wording and prompting. A learned correctness predictor must instead be evaluated against the outcome it purports to estimate, such as benchmark resolution, successful local completion or beneficial remote delegation.

Calibration describes agreement between probabilities and observed frequencies. Guo et al. examine calibration in modern neural networks and motivate held-out calibration procedures such as temperature scaling (Guo et al. 2017). The general lesson applies to routing even though the original classification setting differs from software engineering: a score that orders examples well need not provide reliable probabilities, and a base-rate predictor can be reasonably calibrated while offering little discrimination. Ranking, calibration and final decision quality therefore require separate measurements.

For correctness predictors, reliability diagrams, Brier scores and carefully specified calibration-error summaries are useful complements to discrimination measures. Expected calibration error depends on binning and sample size; a single number can conceal poor behavior in a rare but consequential task stratum. AUROC evaluates a ranking over binary outcomes, while precision-recall can be more informative under severe class imbalance. Neither substitutes for the system-level question of how much accepted work is obtained for a given resource budget. Risk-coverage, resolution-cost and regret curves connect prediction quality to actual routing behavior.

Calibration and decision thresholds must be fitted on data disjoint from final evaluation. Selecting a threshold on the test set to achieve an attractive cost or quality point is part of training, even if model parameters remain fixed. A fixed threshold chosen on calibration data should be evaluated with its realized test exposure; an exact-budget allocation over a scored batch should be described as batch selection. These are different deployment settings.

The local/remote setting adds several sources of distribution shift. New repositories may use different languages, dependencies and issue conventions. Model updates change capability and output style. Quantization, context truncation, retrieval and tool limits can alter success even when the checkpoint name is unchanged. Distilling the student changes the relationship the router is intended to predict. Consequently, a later improvement in the composite system must be decomposed into student changes and router recalibration. A frozen-router comparison and a separately recalibrated-router comparison answer different causal questions.

2.5 Budget-aware routing and cascades

FrugalGPT formulates a budget-constrained cascade whose scorer evaluates a query and generated answer; it learns a sequence of APIs and acceptance thresholds. RouteLLM instead predicts preference for the stronger model before generation, using pairwise preference data and a tunable selection threshold (L. Chen, Zaharia, and Zou 2024; Ong et al. 2025). Its full paper distinguishes reward scoring after generation from routing before generation and evaluates performance recovered relative to the strong/weak gap. These works establish useful baselines and software components. They do not establish that conversational preferences transfer directly to executable repository correctness. The label, task distribution, candidate model pair and feature budget all matter.

The distinction also limits which published metrics transfer unchanged. RouteLLM uses strong-model call share as a cost proxy and normalizes performance by the gap between its candidate models. In an agent loop, individual calls have different token counts and one task can require many calls; a small or negative baseline performance gap also makes normalized recovery difficult to interpret. The thesis therefore retains direct resolution and monetary-cost measurements, while using call share and normalized summaries only with their assumptions stated. FrugalGPT's answer-dependent scorer provides a closer conceptual baseline for verification-triggered escalation, but its query-answering experiments do not establish the cost of repository execution and repair loops.

A prompt-only router acts before generation and pays for one selected model. A cascade first runs a cheaper model, observes its output or execution feedback, and then decides whether to invoke a stronger model. The cascade obtains richer evidence but incurs the cost of the initial attempt even when it escalates. It can also lengthen the remote request by including failed edits, tool traces and summaries. Describing both designs as the same remote-call fraction would hide a meaningful cost difference.

The relevant quantity is total cost per task and per accepted task. For API models, input, cached input, output and provider-specific reasoning charges can differ. For self-hosted models, GPU allocation, idle time, memory requirements, batching and operational overhead determine cost. The local device also consumes electricity and user time even when it generates no cloud invoice. Training, teacher-data generation and artifact distribution add fixed or amortized costs that can dominate at small deployment scale. These terms motivate a price ledger and measured token accounting rather than translating a call-count reduction into an asserted monetary saving.

Fair model comparison requires more than a list of prices. Two systems at the same monetary budget can process different token volumes or make different numbers of tool calls. A model with a low output-token price may generate longer answers or require more repair attempts. Cache behavior depends on request patterns and provider semantics, and idle self-hosted capacity can reverse an apparent advantage measured at full utilization. The proposed study should therefore report both the observed resolution-cost frontier and sensitivity to clearly labelled deployment assumptions.

An oracle router that selects the better observed outcome for each task is useful for measuring remaining opportunity. It is not a deployable competitor and is especially optimistic when selection uses noisy estimates from a finite number of

samples. Random routing at matched exposure or matched expected cost remains essential: it reveals whether the learned allocation adds value beyond simply mixing two systems. A simple interpretable rule provides another baseline against which extra routing complexity must justify itself.

2.6 Speculative decoding as inspiration, with different guarantees

Speculative decoding uses a cheaper draft process to propose tokens that a target model verifies (Leviathan, Kalman, and Matias 2023). Dynamic speculation changes the amount of proposed work instead of using a fixed draft length. The DISCO paper specifies a learned classifier using top draft probabilities, entropy and token position to decide when drafting should stop (Mamou et al. 2024). The related Hugging Face/Intel implementation article also describes a draft-confidence stopping rule. These are related mechanisms with different predictors, not one interchangeable confidence algorithm. Both allocate computation according to the chance that cheap speculative work will be useful.

DeepInfer supplies a more recent implementation of this idea. Its documented DSpark path includes a learned confidence head for draft-token acceptance, cumulative estimates for surviving prefixes, runtime cost curves and confidence-scheduled verification. The scheduler allocates a verification budget across requests and records acceptance and confidence-error statistics (DeepSeek-AI 2026). The inspected source also imposes dependencies on which historical estimates may influence scheduling. These details make the method more specific than a generic instruction to trust a small model when its confidence is high.

The analogy to task routing is productive at the level of resource allocation: estimate useful work, compare expected benefit with cost, and check predictions against outcomes. The acceptance event and its guarantees are different. Token verification can be part of a sampling procedure designed to preserve the target distribution. Accepting an entire locally generated patch after incomplete tests does not preserve the distribution of patches the remote model would have generated, and it does not guarantee semantic correctness. A remote reviewer is itself fallible.

Dependence is also different. A speculative token prefix has a particular sequential structure, whereas software subtasks may share files, assumptions and hidden failure causes. Multiplying marginal confidence scores for several edits does not generally produce a valid probability that their combined patch succeeds. A router may instead predict final task utility directly or use explicitly conditional transition estimates. Either approach requires empirical calibration on the actual decision granularity.

The thesis therefore treats confidence-aware speculative scheduling as a design inspiration, not as a theorem guaranteeing accurate task routing. Token-level speculative acceleration and task-level model selection should be evaluated as separate interventions if both are implemented. The former concerns inference efficiency under a particular serving engine and hardware configuration; the latter concerns which system should perform software-engineering work. Their gains cannot be added without measuring their interaction and overhead.

2.7 Distillation and progressive local capability

Knowledge distillation transfers behavior from a teacher to a student (Hinton, Vinyals, and Dean 2015). In this application the desired effect is increased local task success, enabling the routing policy to rely less often on remote inference at a fixed quality requirement. A reduction in remote usage alone is insufficient evidence of distillation: it could be caused by changing a threshold or accepting more local failures. Student improvement must be established independently on untouched tasks before evaluating the composite system.

Sequence distillation and token-distribution distillation impose different data requirements. Sequence distillation trains on teacher-generated responses, patches or observable tool trajectories. It can proceed without access to teacher logits and is compatible with different teacher/student tokenizers, provided the examples are represented correctly for the student. Token-level distillation typically compares teacher and student predictive distributions, often using a temperature-scaled divergence combined with supervised loss. That comparison requires a meaningful correspondence between token events. A DeepSeek tokenizer and a Qwen tokenizer cannot be assumed aligned merely because both process the same source text. Sparse top-token API outputs also do not automatically supply a full-vocabulary teacher distribution.

Verification changes the training distribution. Selecting teacher outputs that pass tests can improve the reliability of demonstrations, but it favors tasks for which the teacher and available tests succeed. It may omit difficult requests and retain solutions that exploit weaknesses in the test suite. A verification pipeline should therefore preserve selection counts and reasons, separate training verifiers from final evaluation, and maintain an explicit account of the examples that were never eligible. Teacher-generated code should not be treated as correct solely because it came from the larger model.

The inspected DeepSeek-R1 paper describes rule-based accuracy rewards, rejection-filtered training data and SFT-based distillation of smaller models. Its discussion provides a concrete reason to include supervised distillation as a strong baseline before adding more complex reinforcement-learning stages (DeepSeek-AI 2025). Those observations arise from the authors' particular models, data and compute regime. They do not establish that distillation always dominates direct reinforcement learning, that a small customer dataset is sufficient, or that improvement on reasoning questions implies improved repository repair.

The software-engineering setting raises a further transfer issue. A final patch may omit the process needed to discover it: repository search, diagnosis, testing and revision. If the student is intended to run an agent loop, final-answer imitation and trajectory imitation should be distinguished. Observable tool actions and concise task-state summaries can be legitimate training targets; the design does not require access to a provider's private reasoning trace. Repository instructions and retrieval remain lower-cost personalization baselines that should be evaluated before attributing benefits to model training.

2.8 Parameter-efficient adaptation and AI feedback

LoRA represents weight updates with low-rank trainable components while retaining a frozen base model (Hu et al. 2021). QLoRA combines a quantized

frozen base with parameter-efficient adaptation to reduce training memory (Dettmers et al. 2023). These techniques address how parameters are updated; they do not specify which demonstrations are trustworthy or whether the student improves. They also do not remove the memory and compute required for forward passes, optimizer state, activations and evaluation.

The deployment artifact is part of the treatment. An adapter trained in one framework may need merging or conversion before it can be used by the selected local runtime. Quantization can alter outputs, tool formatting and task success. A fair evaluation must include the actual artifact intended for the user's hardware rather than report only unquantized server performance. Base checkpoint, tokenizer, chat template, adapter revision and export procedure should therefore be recorded together.

Direct Preference Optimization learns from preferred and rejected responses relative to a reference policy, providing an alternative to explicitly fitting and optimizing a separate reward model (Rafailov et al. 2023). Reinforcement learning from AI feedback identifies the source of feedback, not one particular algorithm. AI-generated rankings, executable-test preferences and human review are distinct label sources even when supplied to the same optimizer. Constitutional AI combines supervised critique/revision with preference-model training and reinforcement learning; its reported setup retains human helpfulness labels while using AI feedback for harmlessness (Bai et al. 2022). This distinction prevents the mistaken inference that the original method removes every human supervisory input. DeepSeekMath and DeepSeek-R1 motivate verifiable objectives for reasoning tasks (Shao et al. 2024; DeepSeek-AI 2025). None of these sources establishes that an AI judge reliably substitutes for independent code execution.

For this thesis, the controlled sequence is a frozen student, verified-teacher SFT, and only then an optional preference-learning extension with a specified label source. Equalizing base model, data splits, example/token budgets and evaluation conditions helps distinguish the benefit of better supervision from additional compute. If online reinforcement learning is added, candidate-generation and verification costs must be included. Reward hacking, incomplete tests, correlated teacher/judge errors and catastrophic forgetting remain threats requiring separate measurements.

Personal or customer-specific adapters add fragmentation: small datasets may not support robust calibration, and each base/adapter combination needs deployment and rollback management. Training consent, teacher-output rights and artifact licenses are separate from owning the application distribution channel. These considerations motivate an opt-in, versioned data pipeline and a comparison with simpler retrieval/instruction personalization. They are implementation conditions for the study, not evidence that every customer requires an individual adapter.

2.9 Benchmark validity, contamination and harness effects

Executable evaluation is stronger than relying solely on an LLM judge, but its interpretation remains bounded by the tests. EvalPlus demonstrates why test coverage and benchmark revisions matter for code-generation assessment (Liu et al. 2023). The same principle applies to repository repair: a passing patch may

satisfy the selected tests while introducing untested regressions, and a failing run can reflect an environment defect rather than reasoning failure. Infrastructure, extraction, tool-protocol and solution failures should be recorded separately without using post-hoc exclusions to improve a model's score.

SWE-bench Verified is documented as a subset of 500 problems that software engineers confirmed were solvable (SWE-bench Team 2024). That validation improves the evaluation target but does not make every current model comparison contamination-free or every hidden test complete. The inspected SWE-bench documentation also warns that evaluation results are cached by run identifier and instance identifier, rather than changed patch content. Reusing identifiers across treatments can therefore silently reuse an earlier result. Unique run identities, patch hashes and current-run artifact checks are necessary to establish that the intended experiment actually executed.

SWE-bench Pro extends the repository-task setting toward longer-horizon work (Deng et al. 2025). Its currently inspected V2 documentation specifies a changed public set of 642 tasks across 11 repositories, a locked agent phase and regrading of the captured patch in a fresh sandbox (Scale AI 2026). The documented changes from V1 include task instructions, tests, reference patches and images. Consequently, scores from those releases cannot be merged as though they were measurements on one unchanged benchmark. Reference solutions and verifier fixtures must also remain outside the agent's accessible task context.

Public benchmark age and model release date do not identify a model's training exposure with certainty. Time-windowed collections such as LiveCodeBench improve experimental control, but their contamination properties depend on the model checkpoint and the task's prior availability (Jain et al. 2024). Repository-level splits can limit direct transfer of conventions and solutions between router training and testing; temporal splits answer a different generalization question. Both require deduplication and documented grouping rules. Foundation-model pretraining contamination may remain unobservable and should be reported as a residual limitation rather than dismissed by a dataset title.

Finally, published leaderboard accuracy is a system-level observation. Model revision, scaffold, retrieval, tool budgets, test split, number of attempts and reasoning settings can all differ. A pricing table can summarize current offerings, but plotting an incomparable headline score against an unrelated API price does not establish an accuracy-cost frontier. The thesis should distinguish contextual published claims from controlled same-harness measurements and should avoid ranking systems whose evaluation conditions cannot be aligned.

2.10 Synthesis and research contribution

The literature already supplies routing frameworks, deferral objectives, calibrated predictors, agent harnesses, executable benchmarks and efficient adaptation methods. The defensible contribution is their controlled evaluation in a particular setting: whether a local-capable model and a remote model can jointly resolve repository issues at lower total cost while meeting an explicit quality requirement, and whether a router can select that division of work better than simple alternatives.

Three interactions deserve direct empirical treatment. First, confidence and

task difficulty need not identify remote benefit; both models can fail on the same difficult tasks. Second, richer feedback can improve decisions while eliminating savings through local attempts, verification or duplicated context. Third, student adaptation changes both capability and calibration, so apparent improvement in the composite system must be separated into model and routing effects. The experiment's ablations, accounting and statistical plan should correspond to those interactions rather than merely compare the final product with an unspecified LLM baseline.

Literature strand	What it supplies	What the thesis must establish
Selective prediction and deferral	Coverage/risk concepts and an imperfect-expert formulation	Local acceptance and useful delegation on repository tasks
FrugalGPT and RouteLLM	Routing/cascade methods and reusable baselines	Transfer from existing labels and tasks to executable software-engineering outcomes
Calibration research	Methods to relate scores to frequencies	Reliability under repository shift and model adaptation
Speculative inference	Confidence-guided allocation of verification compute	A valid task-level policy without importing token-level guarantees
Distillation and LoRA	Transfer objectives and efficient parameter updates	Held-out student improvement after deployment conversion
Preference and verifiable-feedback learning	Alternative supervision and optimization mechanisms	Gains beyond SFT after counting extra data and compute
SWE benchmarks and agent interfaces	Reproducible tasks, tools and evaluation protocols	Comparable model/hybrid results under one frozen interface

This contribution is conditional and falsifiable. A router that does not outperform a matched-cost baseline, a dual system that violates the quality requirement, or an adapted student that fails to improve is an informative outcome. No such outcome is asserted in this chapter. The proposed methods, source limitations and decision criteria provide the basis on which later measured evidence can support or reject the thesis's claims.

Chapter 3

System design and formalization

3.1 Design principles

The system follows a reuse-first principle. Existing software supplies the agent loop, repository tools, local inference, provider API adaptation, training and benchmark execution. Custom components are introduced only for the research question: recording the decision state, applying a routing policy, measuring costs and joining outcomes to immutable task identifiers. This separation helps avoid comparing an extensively engineered dual system against an artificially weak baseline.

The proposed repository-level harness is mini-SWE-agent, subject to a pinned-version integration check. Its command-oriented interface allows the same repository interaction pattern to be used with multiple model providers. Ollama or llama.cpp is a candidate local deployment runtime; the actual supported model architecture, quantization, chat template and tool behavior must be verified together. A hosted endpoint can initially provide the remote model. Transformers, PEFT and TRL provide the student-training stack. Choosing these components is a design decision, not a claim that all combinations have passed compatibility tests.

The experiment separates four concerns: the policy determines which model to use; the model proposes an action; the harness determines whether and how that action may execute; and the evaluator assesses the final artifact in an independent environment. A model switch does not grant new tool permissions. Tests used by the agent for development are distinguished from hidden benchmark checks used for final scoring.

3.2 State and action spaces

Let task i be an issue description paired with a repository at a fixed base commit. At decision step t, the observable state is

$$s_{it} = (x_i, h_{it}, w_{it}, b_{it}, d_i),$$

where x_i contains the task and permitted repository context, h_{it} is the interaction history, w_{it} records workspace changes and available verification evidence, b_{it} is

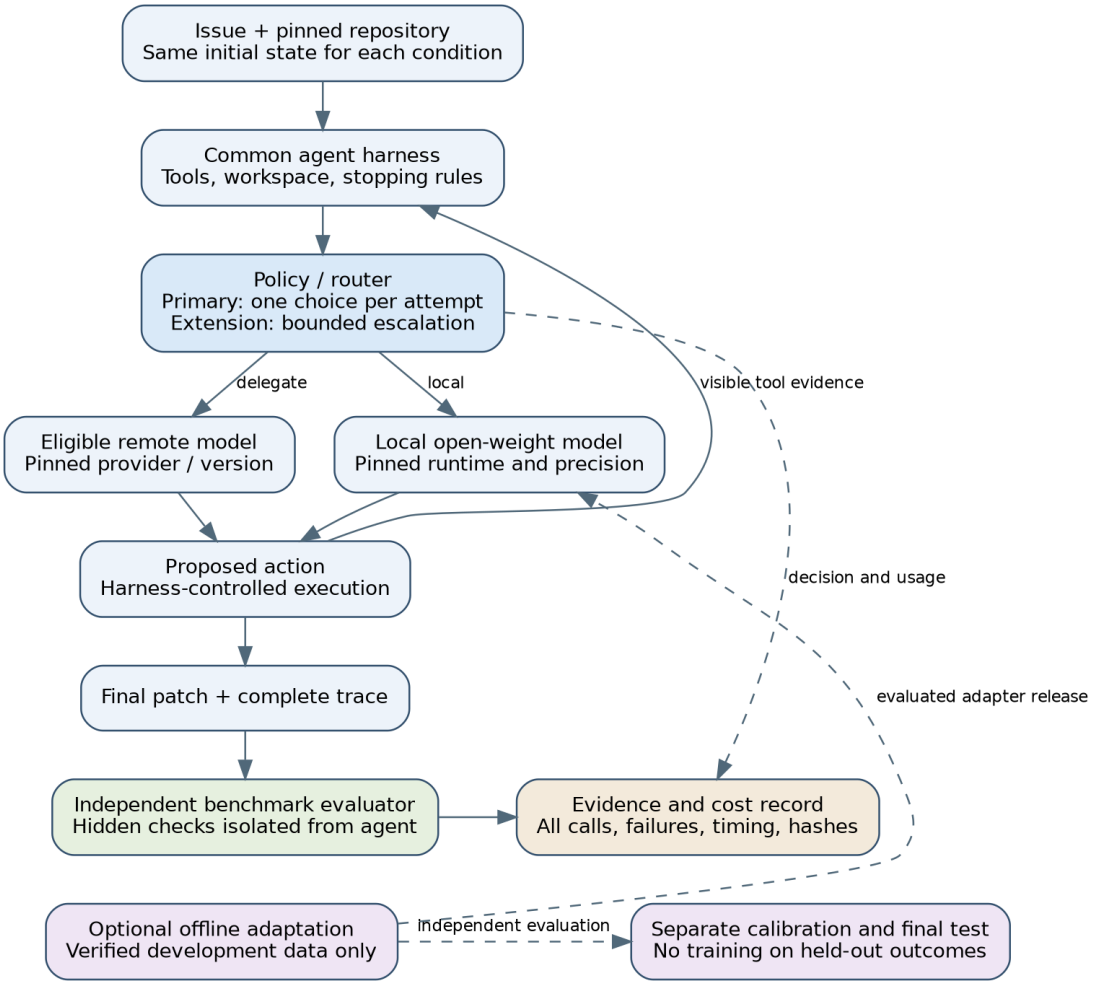


Figure 3.1: Proposed architecture and evidence flow. Arrows indicate interfaces, not measured performance.

the remaining budget, and d_i describes the eligible deployment configuration. The router receives a documented feature function $\phi(s_{it})$. It cannot access hidden evaluation tests, reference patches, future tool results or the eventual resolution label.

The minimal action set is $\mathcal{A} = \{L, R, \text{stop}\}$, where L and R request local or remote continuation. A sequential extension may include an explicit verification action V. Its benefit and cost must be assessed as an action; verification is not a free source of confidence. The initial confirmatory policy is kept simple enough to attribute differences to routing. Richer orchestration, parallel branches and multiple simultaneous agents are separate experiments.

A policy $\pi(a \mid s)$ chooses an action subject to hard constraints. These include user/provider permissions, model capability, context capacity and remaining budget. Confidence cannot override a prohibition on sending a repository to a remote provider. When no permitted action fits the remaining budget, the task terminates with a recorded unresolved or budget-exhausted outcome rather than silently changing the experimental conditions.

3.3 Estimating benefit rather than asking for confidence

Let $p_m(x)$ be the conditional probability that a bounded run with model m resolves the task under the common harness. A pre-call router can estimate

$$\Delta p(x) = p_R(x) - p_L(x).$$

It may prefer remote execution when the expected benefit exceeds an incremental-cost penalty:

$$\widehat{\Delta p}(x) > \lambda \, \widehat{\Delta C}(x),$$

with λ chosen using development data according to the prespecified cost or quality constraint. This expression does not assert that resolution and cost are perfectly predictable. It defines a decision objective and exposes the quantities requiring calibration. A small classifier, an embedding-based predictor or an existing routing model can instantiate it; a separate generative router is not required by the architecture.

The local-failure comparator ranks by $1 - \hat{p}_L(x)$. It is informative because it tests the intuitive hypothesis that local uncertainty is enough. The benefit predictor instead asks whether remote execution is expected to change the outcome. The latter objective can direct remote resources toward tasks with a realistic chance of improvement rather than those that are simply difficult. If the product values particular classes of requests, their utility weights must be declared before evaluation rather than inferred after seeing which tasks the policy serves.

Calibration is an empirical relationship between predictions and outcomes. A statement such as “I am 90% confident” is not accepted as a calibrated probability. Token log probabilities, hidden-state features and self-evaluation outputs are candidate features, not correctness guarantees. Their acquisition overhead is recorded, and a feature is excluded if it requires information unavailable at the policy’s decision point.

3.4 Sequential delegation and state continuity

A local-first cascade differs from pre-call selection. It first pays for a local attempt, observes its permitted evidence and then decides whether to escalate. If the local attempt changes files, the remote continuation must receive a well-defined state: base commit, current patch, task objective, executed commands, outcomes and remaining budget. Unfinished tools are completed or cancelled before switching. The system does not attempt to transfer incompatible key–value caches or opaque provider reasoning representations.

The cost of an unsuccessful local attempt remains part of the dual system's cost. Remote repair can also require extra context, and its probability of success may differ from starting remotely. The thesis therefore distinguishes $p_R(x)$ from $p_R(s_{it})$ after local work. Assuming that the same remote-only outcome can simply be substituted after any local failure would erase a central system effect.

For paired comparisons, every experimental condition starts from the same immutable repository snapshot. Workspaces, generated files and cached agent conclusions are isolated by task, seed and condition. Dependency caches may be shared only when they do not expose solutions or mutable task state. Replaying a tool event is not equivalent to re-executing it: external side effects, generated dependencies and process state require explicit treatment.

3.5 Confidence-scheduled verification as an analogy

DeepInfer provides a concrete implementation example of using learned confidence estimates to allocate inference computation. Its DSpark mechanism predicts draft-token acceptance and schedules verification using acceptance estimates and runtime cost information. This motivates a general design principle: spend computation where it is predicted to produce useful progress, and compare predicted benefit with observed behavior.

The analogy has limits. Token-level speculative decoding involves a particular relationship between a draft distribution and target verification. A task router that accepts a local patch does not inherit that algorithm's distribution-preservation properties. Repository tests are incomplete, remote reviews can be wrong, and task failure events are correlated. Multiplying arbitrary subtask confidence scores as if they were independent is unjustified. The proposed system uses direct task/continuation outcome labels and reports uncertainty rather than importing a token-level guarantee.

3.6 Student learning mechanisms

The adaptation study begins with sequence distillation. For training task i, a teacher generates an observable solution or trajectory, which is retained only if it passes the declared verification procedure. The student is trained on the resulting target sequence y_i^T:

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\sum_i \sum_{t \in \mathcal{T}_i} \log p_\theta(y_{it}^T \mid x_i, y_{i,<t}^T),$$

where $\mathcal{T}_i$ contains completion tokens included in the loss. Prompt tokens are masked. LoRA changes which parameters are optimized; it does not determine whether the targets are correct or representative. Complete examples that exceed the declared context limit are explicitly handled instead of silently truncating a solution and treating the remainder as valid supervision.

Verification filtering creates selection effects. Tasks with no verified teacher solution disappear from the training set, while the evaluation corpus must retain its prespecified eligibility rules. A student may consequently receive a distribution biased toward tasks the teacher already solves. Counts, task classes and exclusion reasons are recorded. A token- and compute-matched adaptation control is needed to distinguish teacher-data quality from the mere effect of extra training.

A token-distribution arm has a different objective. With aligned teacher/student tokenization, an example formulation is

$$\mathcal{L}_{\mathrm{KD}} = \alpha \mathcal{L}_{\mathrm{SFT}} + (1-\alpha)\tau^2 \sum_t D_{\mathrm{KL}}\big(p_T^\tau(\cdot \mid c_t) \,\|\, p_\theta^\tau(\cdot \mid c_t)\big)\,.$$

Temperature τ, mixture coefficient α, context c_t, vocabulary mapping and teacher precision must be specified. Different Qwen and DeepSeek tokenizers cannot be assumed to share token events. A same-family teacher/student pair is a practical controlled case, but still requires exact vocabulary, special-token and template checks. A provider's limited top-k log probabilities are not the full teacher distribution; treating the omitted probability mass as zero changes the objective and requires its own method and ablation.

RLAIF describes the use of AI-generated feedback. It is distinct from using executable tests as a verifiable reward. DPO on teacher-ranked pairs is one possible preference-learning arm; DPO on pass/fail pairs is verifier-based preference learning. Online reinforcement learning adds generation and reward-computation costs and is a separate extension. The experiment does not relabel ordinary supervised fine-tuning as RLAIF.

3.7 Adaptation and routing as separate interventions

An adapted student changes the distribution of errors on which the router was trained. The experiment evaluates both a frozen router with the adapted student and a newly calibrated router fitted without final-test labels. This forms a factorial distinction between student capability and routing adaptation. Otherwise, an apparent gain in local coverage could be attributed incorrectly to the language model when it arose from threshold changes.

Model/adapter releases are immutable artifacts with hashes, base revision, tokenizer revision and training provenance. The exact deployed artifact—including quantization or adapter merging—is reevaluated. Training-time loss alone is not a software-engineering outcome, and improvements before export do not guarantee the same behavior in a local runtime.

3.8 Cost model

For model call j, API expense is recorded using the provider's billed categories:

$$C_j^{\text{API}} = \frac{r_u n_u + r_c n_c + r_o n_o + r_w n_w}{10^6} + f_j,$$

where n_u, n_c, n_o, n_w represent uncached input, cached input, output and any separately billed cache-write units, with their respective rates. A category is omitted or reformulated when a provider bills differently. Reasoning/thinking tokens are included according to the actual invoice definition; they must not be counted twice as both visible output and an independent category. Tool, search, batch, regional and context-tier charges are stored explicitly when applicable.

The complete task cost includes all model calls, routing, verification, execution and relevant allocations:

$$C_i = \sum_{j \in i} C_j^{\text{model}} + C_i^{\text{router}} + C_i^{\text{sandbox}} + C_i^{\text{verification}} + C_i^{\text{storage/network}} + C_i^{\text{allocated}}.$$

For self-hosted models, GPU rental is charged for provisioned time, including loading, idle periods and shutdown delay. Local operation is reported under two accounting views: marginal cash expenditure and total economic cost including energy and hardware allocation. A zero API rate is therefore not represented as zero total cost. Cost per resolved issue has the total cost of all assigned attempts, including failures, in its numerator and the number of resolved issues in its denominator. Under repeated sampling, the numerator and denominator use the same registered aggregation. Zero resolved issues makes this ratio undefined; unsuccessful runs cannot be removed from the invoice.

Distillation adds an amortization question. If training and data generation cost C_{adapt} and the adapted system saves an expected s per task at an acceptable quality level, an illustrative break-even task count is $N = C_{\text{adapt}}/s$ for $s > 0$. No break-even claim is valid when quality is not maintained or the estimated saving is nonpositive. Estimates must be accompanied by workload assumptions and uncertainty rather than presented as a guaranteed product margin.

3.9 Evidence record

Every run records task and repository identifiers, base commit, dataset/harness revisions, container image digest, condition, model/adapter revision, decoding settings, seed where supported, resource limits, prompts/tool traces under the retention policy, output patch, evaluator logs and billed usage. Cached and uncached scenarios are identified separately. A run manifest links these records through hashes and identifies whether a stage was prepared, started, completed, failed or unrun.

This evidence structure serves both scientific reproducibility and task continuity. Jira tickets can index work and Confluence can host the manuscript, but neither substitutes for run manifests and versioned artifacts. If external tools are unavailable, local state records remain explicitly pending synchronization and must not invent issue keys, page IDs or completed runs.

Chapter 4

Model and price landscape

This chapter freezes the **model-selection evidence on 25 October 2026**. It contains published vendor claims and a dated price catalog, not results from the TTGL experiment. "Latest" means the latest relevant release found in the inspected official repositories and model cards; this is not a claim of exhaustive market coverage. The accompanying model catalog records exact identifiers, model revisions, licenses, service names, units, and evidence status. The source register preserves retrieval timestamps, content hashes, revision-pinned URLs, selected source excerpts, and unsuccessful pricing requests.

4.1 Candidate selection

The current Qwen release catalog lists Qwen3.8-27B, released on 14 August 2026, and Qwen3.8-2.4T-A95B, released on 12 August. Its most recent sub-10B dense releases are **Qwen3.5-0.8B, 2B, 4B, and 9B**, released on 2 March. Consequently, the local-first study should start with **Qwen3.5-4B**, use **Qwen3.5-9B** for a capacity ablation, and assess **Qwen3.8-27B** as a feasible self-hosted teacher candidate. Calling a small local checkpoint "Qwen3.8" would misidentify the verified release. Qwen2.5 remains useful for reproducing the existing implementation, but it is a historical baseline rather than a current-generation claim.[1]

The ten candidates below cover local inference, remotely hosted open weights, inexpensive closed APIs, and premium closed APIs. Candidate status does not imply that TTGL already supports the model or that an account can access it.

Candidate	Proposed role	Parameters: total / active	Weight access and license
Qwen3.5-4B	Primary local student	4B / dense	Open weights, Apache-2.0
Qwen3.5-9B	Local capacity ablation	9B / dense	Open weights, Apache-2.0
Qwen3.8-27B	Self-hosted teacher or remote baseline	27B / dense	Open weights, Apache-2.0
DeepSeek-V4.1-Flash-1017	Remote teacher candidate	Base family: 552B backbone; 8B prefill / 16B decode active	Open weights, MIT

[1] Qwen official release catalog. Individual weight-card revisions and licenses are recorded in `model-catalog.csv` and `market-sources.json`.

Candidate	Proposed role	Parameters: total / active	Weight access and license
GLM-5.3-Flash	Low-price remote baseline	320B / 18B	Open weights, MIT
Kimi K3	Remote frontier reference	2.8T / 104B	Open weights, custom Kimi K3 License
GPT-6.1 Sol	Premium closed baseline	Undisclosed	Closed weights
GPT-6 Luna	Inexpensive closed baseline	Undisclosed	Closed weights
Claude Sonnet 5.5	Premium closed baseline	Undisclosed	Closed weights
Claude Opus 5.5	Higher-price closed reference	Undisclosed	Closed weights

The official OpenAI and Anthropic SDK model catalogs identify the closed-model API names `gpt-6.1-sol`, `gpt-6-luna`, `claude-sonnet-5-5`, and `claude-opus-5-5`. An SDK entry establishes an identifier, not an immutable weight revision, capability guarantee, or account entitlement. Freeze the returned serving revision and all provider settings at execution; do not silently roll an alias forward midway through a comparison.[2]

Open weights and open-source software are different properties. The Qwen licenses above were checked in the individual weight repositories. GLM-5.3-Flash weights use MIT even though its umbrella GLM-5 code repository uses Apache-2.0. Kimi K3 applies its custom license to both code and weights: among other provisions, it requires a separate commercial agreement for a model-as-a-service business above the specified aggregate revenue threshold and branding for certain very large services. The exact license includes exceptions and definitions; it must be read for the intended deployment. A permissive harness or SDK license does not license the model, and a model-weight license does not establish hosted-API output-training rights.[3]

The parameter distinction materially changes feasibility. GLM's 18B active parameters do not mean only 18B weights need storage. Even an idealized 4-bit representation of its 320B total parameters requires approximately 160 GB before scales, activations, KV cache, and runtime overhead. Kimi's 2.8T parameters require approximately 1.4 TB at that idealized precision. DeepSeek's base card specifies a 552B backbone and separately describes 196B Engram conditional memory; the inspected wording does not establish an unambiguous combined total, so this chapter does not add them. The backbone alone is already too large for full residency on one 80 GB GPU. Qwen3.8-27B has approximately 54 GB of raw BF16 weights, making a memory-profiled single-GPU inference trial plausible; that arithmetic is not proof of long-context or training feasibility.[4]

[2] OpenAI SDK model identifiers; Anthropic SDK model identifiers. Retrieved 25 October 2026.

[3] Qwen 4B weight license, 9B weight license, and 27B weight license; GLM-5.3-Flash weight license; Kimi K3 license; DeepSeek-1017 license statement.

[4] GLM official model table; Kimi K3 architecture; DeepSeek base family architecture.

4.2 Price comparison and what it measures

All direct-API rates in the following table are provisional secondary observations, not vendor-verified quotes. They come from one revision-pinned LiteLLM community catalog snapshot. Direct OpenAI, Anthropic, DeepSeek, Nebius, and Z.ai pricing pages were requested and returned proxy `403` errors, including a later recheck after model-card access succeeded. Successful Hugging Face retrieval therefore does not establish access to those billing sources. The exact failures are recorded rather than replaced with guessed prices.[5]

Rates are **USD per one million text tokens**, in the order **uncached input / cache-read input / output**, for the catalog's ordinary base tier. They exclude cache creation, long-context multipliers, batch discounts, priority or fast modes, regional adjustments, tool charges, taxes, and subscription conditions. Missing cache prices are unknown, not zero.

Model / priced service	Input	Cache read	Output	Qualification
Qwen3.5-4B / local	—	—	—	No API token bill; hardware, energy, and training still cost money
Qwen3.5-9B / local	—	—	—	Same accounting boundary
Qwen3.8-27B / Nebius Token Factory	$0.45	Unknown	$3.00	Third-party service; not a Runpod rental quote
DeepSeek-V4.1-Flash / Tencent Cloud	$0.30	$0.006	$1.20	Alias-to-1017 mapping unverified; not direct DeepSeek pricing
GLM-5.3-Flash / Z.ai	$0.15	$0.03	$0.50	Base catalog rate
Kimi K3 / Moonshot	$3.00	$0.30	$15.00	Base catalog rate
GPT-6.1 Sol / OpenAI	$2.00	$0.10	$10.00	Standard tier, input at most 272K tokens
GPT-6 Luna / OpenAI	$0.10	$0.01	$0.50	Standard tier, input at most 272K tokens
Claude Sonnet 5.5 / Anthropic	$2.00	$0.10	$10.00	Standard text tier; separate cache-write charges
Claude Opus 5.5 / Anthropic	$4.00	$0.20	$20.00	Standard text tier; separate cache-write charges

An independently retrieved **primary GitHub Copilot pricing file** lists the same four closed-model base input, cache-read, and output rates. This corroborates those numbers for the Copilot channel only; it does not certify the direct-provider API contract. Its service plan, credit accounting, and model availability are separate. Neither source establishes a free API tier or the price of an available 80 GB Runpod instance.[6]

This price table motivates inclusion of an inexpensive closed baseline: a dual system must compete with an already cheap API, not only a premium teacher. It does not establish cost effectiveness. A task may use many model calls, cached and uncached context, reasoning tokens, failed tools, and retries. The experiment

[5] Secondary source: LiteLLM price catalog, commit dddbaa8, retrieved 25 October 2026. Exact catalog keys, raw selected entries, and the catalog's cited upstream URLs are preserved in `market-sources.json`. Upstream direct API rates remain unverified.

[6] Primary source for its own service channel: GitHub Copilot model prices, retrieved 25 October 2026. This is not a direct-provider API quote.

must compute total inference and tool cost per attempted task, total cost per resolved issue, and wall-clock latency from actual traces. Local cost should be reported both as marginal energy plus incremental infrastructure and under an explicit amortized hardware scenario. Report distillation/training expenditure separately and amortize it over declared future workload sizes; a "free local" product tier is a billing choice, not zero economic cost.

4.3 Published accuracy evidence: contextual, not a leaderboard

Published scores establish that candidate models have been evaluated on relevant tasks. They cannot supply the missing TTGL comparison. Every score below is a **vendor-reported claim** from a primary artifact; none has been reproduced here, and none is a TTGL outcome. A blank latest-model score means insufficient inspected evidence, not zero capability.

Model	Published claim	Harness and limits disclosed by that source
Qwen3.5-4B	LiveCodeBench v6: 55.8	Coding-problem benchmark, not repository-level SWE; exact task count and harness not stated in the inspected card
Qwen3.5-9B	LiveCodeBench v6: 65.6	Same qualification; quantized local deployment must be measured separately
Qwen3.8-27B	DeepSWE v1.1: 42.2; SWE-bench Pro: 61.7	Claude Code; temperature 1.0, top-p 0.95, 256K context; Pro uses vendor-corrected tasks, not an unchanged official task set
DeepSeek-V4.1-Flash-1017	DeepSWE v1.1: 77.0	Mini-SWE-V2; task count, exact harness commit, and benchmark-specific sampling budget not stated in the inspected update card
GLM-5.3-Flash	DeepSWE v1.1: 63.4	mini-swe-agent, temperature 0.95, top-p 1.0, 400K context, six-hour timeout; task count and exact harness commit unspecified
Kimi K3	DeepSWE v1.1: 67.3	README footnote attributes this to mini-SWE-agent; main-table 67.5 uses Kimi Code; task count and exact harness commit unspecified
Four latest closed candidates above	Not established by inspected SDK/pricing sources	Catalog presence and prices provide no SWE accuracy evidence

The Qwen figures come from its exact model cards, the GLM figure from a pinned official chart plus its model-card evaluation footnote, and the Kimi and DeepSeek figures from their official release artifacts.[7] These are different benchmark families and operating points. LiveCodeBench is not SWE-bench Verified; SWE-bench Pro is not DeepSWE; corrected tasks are not the original evaluation set. Even the DeepSWE rows differ in context, timeout, harness, and version details. The table is deliberately **not sorted by score**, and no price–accuracy Pareto frontier is inferred from it.

The parent DeepSeek-V4.1-Flash card illustrates why harness control matters: it reports DeepSWE v1.1 scores ranging from 65.5 with OpenCode to 74.2 with mini-SWE for the same model, under a described eight samples per task, one-million-token context, and 500-step setting. The newer 1017 card reports 77.0 with Mini-SWE-V2; this cannot be treated as a clean model-only improvement over the older number. It also compares older closed releases, including GPT-5.6 Sol and Opus 5; those scores must not be reassigned to GPT-6.1 Sol or Opus 5.5.[8]

For the thesis, "software-engineering accuracy" should mean issue resolution by the benchmark's official executable evaluator. Compare always-local, always-remote, and routed systems on the **same held-out issues, harness revision, tools, prompts, retry policy, step limit, context policy, and task timeout**. A direct single completion without tools is a separate model-only task condition; it must not be presented as equivalent to an agent using shell access and tests. The closed and open models require this same evaluation before any capability or price–accuracy ranking is defensible.

4.4 Runtime readiness and the first bounded study

The verified releases are **research candidates, not deployment-ready TTGL configurations**. The existing Qwen2.5 training stack's Transformers 4.56.2 / TRL 0.23.1 pins do not establish support for Qwen3.5's hybrid architecture or Qwen3.8. The Qwen3.5-4B card recommends current frameworks, illustrates Transformers installed from `main` with serving support, SGLang from its Git repository, and vLLM with a nightly-wheel index. It specifies `qwen3` reasoning parsing and `qwen3_coder` tool parsing, and offers a language-model-only serving mode. It does **not** give a stable numeric minimum version that this chapter can certify. The Qwen3.8-27B card likewise recommends current SGLang, vLLM, or TokenSpeed recipes without specifying a numeric minimum in the inspected text.[9]

Resolve compatible released versions or immutable commits in a new lockfile; verify loading, tool calls, loss masking, one optimizer step, adapter save/reload, and evaluation on an excluded smoke-test task before a paid run. Serving support is not proof of fine-tuning or TRL compatibility. Keep model revision, tokenizer/template revision, quantizer, engine build, precision, reasoning-effort setting, and maximum context in the run manifest. For local Qwen, evaluate the same quantized deployment that the product intends to ship.

[7] Qwen model cards: 4B, 9B, 27B; GLM official benchmark chart and evaluation settings; Kimi README footnotes; DeepSeek-1017 model card.

[8] DeepSeek-V4.1-Flash base release card, especially "Performance across agent scaffolds"; this is separate from the 1017 update card.

[9] Qwen3.5-4B serving instructions and Qwen3.8-27B serving instructions. Framework recommendations are vendor documentation, not a completed TTGL compatibility test.

DeepInfer is valuable related work, but its current README targets **NVIDIA SM100 or Ascend 950**, requires Linux, GCC at least 13, Python 3.12, and CUDA 13 or CANN 9.2, and states that a complete service still needs KV-cache management, disaggregation, and load balancing. Its example benchmarks use four SM100 GPUs for prefill and 64 for decode. This is not the first experiment's single A100/H100-class 80 GB deployment. Its default checkpoint is exactly DeepSeek-V4.1-Flash-1017. Use the model through a suitable remote service for the bounded pilot if exact revision and API behavior can be established; use a manageable Qwen teacher for direct access to full logits.[10]

With a US$100 total GPU budget, the ten-model catalog is a selection space rather than a commitment to an exhaustive training matrix. Prioritize one local student, its always-local baseline, one remote teacher baseline, and a routed combination. Add the 9B capacity ablation only after a costed smoke run establishes feasibility. An inexpensive closed API is a useful later comparator, subject to a separate authorized API budget; the Runpod approval does not cover that spending. Keep broad frontier comparisons descriptive until funded comparable evaluations exist. Teacher API text or top-k log probabilities must not be described as full logits; exact-token distillation additionally requires tokenizer compatibility or an explicitly validated alignment method. All held-out software-engineering accuracy, router calibration, savings, and distillation gains remain unmeasured.

[10] DeepInfer README, retrieved 25 October 2026; content hash and retrieved source URL in `market-sources.json`. Hardware and complete-service limitations are explicitly documented by the project.

Chapter 5

Published open- and closed-weight software-engineering accuracy

A follow-up search on 25 October 2026 inspected the official SWE-bench site's complete published leaderboard and the complete, non-truncated file tree of its submission repository. The published board contained 180 Verified, 84 Lite, 24 Test, 22 Multimodal, and 13 Multilingual entries. No identities matching **GPT-6.1 Sol, Claude Sonnet 5.5, Claude Opus 5.5, or Gemini 3.1 Pro** appeared in the inspected leaderboard or submission paths. This documents a limitation of the inspected evidence, not proof that no evaluation exists anywhere. In particular, the Gemini 3 Pro entry identifies `gemini-3-pro-preview`; its score cannot be relabeled as Gemini 3.1 Pro.[1]

The benchmark does provide an attributable **historical comparison of open and closed models** using the same named harness release. The following records are dated **17 February 2026**, use **mini-SWE-agent 2.0.0**, and specify **one attempt per issue on SWE-bench Verified**. Each selected leaderboard entry contains 500 per-instance outcomes; summing its published resolution flags reproduces the reported numerator and percentage. This consistency check verifies transcription of published records; it is not a new model experiment or independent rerun of the benchmark.

Historical model	Weight access	Reasoning effort	Resolved / 500	Resolution rate
Claude Opus 4.6	Closed	Unspecified	378 / 500	75.6%
GPT-5.2	Closed	High	364 / 500	72.8%
Claude Sonnet 4.5 (`20250929`)	Closed	High	357 / 500	71.4%
GLM-5	Open	High	364 / 500	72.8%
Kimi K2.5	Open	High	354 / 500	70.8%
MiniMax M2.5	Open	High	379 / 500	75.8%

[1] Official SWE-bench leaderboard JSON, site revision `193160a463a435d05cf44a1fa9dc5eac832113c3`; official submission repository, revision `40f164d5b8f1d249bf95a6df8b74b577fd8e519d`. Retrieved 25 October 2026.

These are primary **benchmark-published submission records**, with per-instance details and linked trajectories, rather than scores copied from a competing vendor's comparison chart. Weight-access classifications and reasoning settings are those of the benchmark records. The selected records have `checked: null`; that must not be rewritten as an independently audited result. Shared task set and harness version improve comparability, but the inspected metadata does not establish identical sampling, context, tool budgets, serving revisions, or reasoning settings. Therefore this is historical descriptive evidence, not a causal ranking of model families, proof of equivalence, or evidence that an open model beats a current closed model. The single-issue difference between two records is not a statistical significance claim.[2]

The comparison supports a concrete research motivation: the benchmark's older open-weight and closed-weight systems both resolved substantial fractions of the same repository-level task set, making measured cost and routing behavior useful questions. It does **not** establish the accuracy of Qwen3.5-4B, Qwen3.8-27B, DeepSeek-V4.1-Flash-1017, GPT-6.1 Sol, or Claude 5.5. Historical resolution percentages must not be combined with current token prices to construct a purported current-market Pareto frontier. The thesis's current-model comparison still requires its preregistered common-harness evaluation.

A newer Google submission supplies another clearly identified contextual point: **Gemini 3.5 Flash with mini-SWE-agent 2.4.2**, dated **1 September 2026**, reports **359 / 500 = 71.8%** on SWE-bench Verified. Its README states that it used the bundled `swebench.yaml` unchanged, one attempt per instance, and that 441 instances were attempted while 59 produced no patch. Its denominator remains 500. The metadata specifies medium reasoning effort and harness commit `2f6637a2557da1f7b5f2b583ab7090a077bbeed2`. This submission exists in the experiment repository but was absent from the inspected generated website board, illustrating publication lag. It is a different model and harness from the historical table and is not evidence for Gemini 3.1 Pro or the latest Google release.[3]

The accuracy evidence register contains revision-pinned source URLs, SHA-256 hashes, selected raw submission metadata, exact record names, and the search scope. No TTGL accuracy or routing outcomes are introduced by this chapter.

[2] Pinned official submission metadata: Opus 4.6, GPT-5.2, Sonnet 4.5, GLM-5, Kimi K2.5, MiniMax M2.5.

[3] Gemini 3.5 Flash submission README and metadata.

Chapter 6

Experimental methodology

This chapter specifies a prospective study of a local–remote coding system. It defines the experiments, estimands, controls, measurements and analysis before new software-engineering outcomes are collected. Numerical thresholds in this chapter are design choices, not findings. The existing TTGL implementation provides data-validation, routing, training-preparation and generation components for a coding-problem study; it has not yet integrated or executed the repository-level agent evaluation specified here. An implementation is not treated as an evaluated treatment until its configuration and outputs pass the checks below.

6.1 Research questions and confirmatory claims

The central question is whether a system combining a small local-capable language model, a stronger remote model and an inexpensive router can resolve software issues at lower total cost while retaining an acceptable fraction of the stronger model's performance. This question has two components: whether the pair contains useful complementary capabilities, and whether an implementable router can identify where the remote model contributes enough value to justify its cost. A large gap between a router and an outcome-informed oracle would indicate an allocation problem; a small oracle advantage over the local model would indicate limited value in that model pair on the selected workload.

The study addresses five research questions. RQ1 asks whether the dual system is noninferior to the remote-only system in issue-resolution rate and cheaper in mean total cost. RQ2 asks whether learned routing outperforms random and simple heuristic allocation at comparable resource budgets. RQ3 asks how routing quality changes with task distribution, model pair, repository familiarity and local hardware. RQ4 asks whether verified teacher-output adaptation improves the student and shifts the accuracy–cost frontier. RQ5 asks whether conclusions survive comparison with contemporary open-weight and closed-model baselines under a common agent scaffold.

Two claims form the confirmatory sequence for one model pair and one frozen routing policy. First, the dual system's mean resolution rate must be no more than five absolute percentage points below remote-only. Second, conditional on establishing that noninferiority claim, the dual system must have lower mean total task cost. The five-point margin is a proposed engineering tolerance that must be accepted before the final protocol is frozen; it is not a consequence of benchmark

statistics or an assertion that all customers consider that loss acceptable. If stakeholders require a smaller margin, the change must precede the confirmatory run and the sample-size assessment must be repeated.

Let A_D and A_R denote dual and remote-only task-resolution probabilities, and let C_D and C_R denote their mean total costs. The first null hypothesis is $H_{0,A} : A_D - A_R \leq -0.05$. The second is $H_{0,C} : C_D/C_R \geq 1$. A lower one-sided 95% confidence bound above -0.05 supports the first claim; an upper one-sided 95% confidence bound below one supports the second. The fixed gatekeeping order prevents a lower-quality system from being declared successful solely because it spends less. Both estimates and their intervals are reported even when the gate is not passed. Additional model pairs, router architectures and cost points are exploratory unless separately registered with an explicit multiplicity plan.

6.2 What "single LLM" means in this comparison

A model without tools and a coding agent with repository access do not receive equivalent opportunities to solve a software issue. The primary single-model baselines therefore use the same agent scaffold as the dual system. The difference is whether every model request is served by one fixed backend or whether the registered policy can choose between backends. Repository access, shell execution, test commands, edit mechanisms, initial instructions and termination rules remain identical within a comparison.

A bare-LLM condition may be added as a secondary diagnostic. It receives the issue and a fixed, automatically selected repository-context package and returns a patch without interactive tools. It is explicitly labelled a different interface condition. Its contrast with an interactive agent estimates the combined effect of feedback, tool access and repeated inference; it cannot isolate routing or establish that two models are intrinsically superior to one. The context selector is frozen, does not consult the reference patch, and is shared across bare-model conditions.

The study also separates pre-call routing from escalation. In pre-call routing, the router selects one backend for an entire agent attempt before that backend generates an answer. In escalation, a local agent performs bounded work before a remote agent continues from its state. Escalation incurs the local attempt's cost and creates a different context for the remote model. An archived remote-only trajectory is not a counterfactual for that continuation. Pre-call routing is the initial primary treatment because it admits cleaner paired measurements; a real escalation treatment requires fresh executions and is a secondary experiment.

6.3 Benchmark selection and target population

The primary benchmark is SWE-bench Verified, the documented subset of 500 repository issues whose solvability was examined by software engineers (Jimenez et al. 2024; SWE-bench Team 2024). A task comprises an issue statement, a repository and base revision, and an evaluation procedure for the produced patch. The primary outcome is the benchmark evaluator's resolution decision. This measures success against the specified issue tests and regression checks, not general maintainability, security, review acceptance or production fitness.

The selected SWE-bench dataset revision, task-repository revision, evaluator revision and container image digests are fixed together. The current repository documents a version-five command interface and a separate task repository; historical command examples must not be mixed with that interface without checking compatibility. The evaluator's reference-patch check is used on a small preflight subset to establish that images and tests work. Reference patches, hidden test additions and evaluator internals remain outside the agent's accessible workspace. The preflight checks validate the environment, not the candidate model.

The full Verified set offers an interpretable common benchmark, but it is old enough to be vulnerable to pretraining exposure. "Verified" establishes neither model novelty nor absence of contamination. Model-card training-cutoff claims, release dates, public benchmark exposure and any retrieved issue/patch overlaps are documented. No automatic decontamination procedure can prove that an opaque provider never trained on a task. Accordingly, the thesis distinguishes controlled within-benchmark comparisons from claims about unseen customer repositories.

A second benchmark is reserved for external replication after the main method is frozen. SWE-bench Pro is a suitable candidate because its authors provide repository-level, longer-horizon tasks. Its current public version-two documentation specifies 642 tasks across 11 repositories in Harbor format and separates that version from the legacy version-one pipeline (Deng et al. 2025; Scale AI 2026). Any use of this corpus must fix the version, image digests and locked evaluation protocol. It must not pool version-one scores with version-two outcomes or assume the public split duplicates a private leaderboard. External replication will proceed only after repository/task overlap and access requirements are audited; it is not part of the currently funded feasibility pilot.

LiveCodeBench or comparable coding-problem datasets may be used to debug feature extraction, model endpoints and training mechanics. They are not the primary evidence for repository-level software engineering. Existing TTGL coding-problem artifacts are retained as engineering history and are not inserted as the missing results of this prospective study. Examples or outcomes already examined while developing the method are classified as development material.

6.4 Development data, test isolation and repository familiarity

Router development requires observed outcomes from both candidate backends on the same tasks. These paired runs are collected on development issues that are excluded from the final Verified evaluation. Candidate development data may come from non-Verified SWE-bench tasks or a separately licensed issue corpus. Subtracting Verified identifiers alone is insufficient: the preparation pipeline must also remove duplicated issue text, related pull requests, shared reference patches, and clearly equivalent bug instances. Exclusion rules are documented and applied before model comparison. Ambiguous clusters are kept together or excluded from development.

The study reports two distinct generalization estimands. The primary full-Verified comparison is issue-disjoint: none of its task outcomes or solutions may train,

calibrate or select the router. Some repositories can nevertheless be familiar through different development issues. A second, predefined repository-unseen subset of Verified is selected using repository identity and task metadata before any candidate outcomes are inspected. All development issues and adaptation examples from those repositories are excluded. Results for that subset estimate a narrower form of cross-repository transfer. Scores from a subset are not labelled as full-500 leaderboard results.

The repository allocation is established in a signed or hash-recorded manifest. Among remaining eligible development repositories, training and calibration groups are separated by repository when feasible; when too few repositories remain, a within-repository chronological split is used and identified explicitly as the weaker design. The task manifest includes repository, issue identifier, issue timestamp, base commit, related-issue cluster, split and exclusion reason. Test task IDs may be known to orchestration and the evaluator but are not predictor features. The router must not receive the official resolution label, a reference patch, hidden-test feedback or a benchmark-specific difficulty label at decision time.

After the primary test is opened, no feature, model, calibration method, threshold or utility weight is revised on the basis of its outcomes. Any subsequent change defines a new exploratory version and requires a new untouched test set for confirmatory claims. This includes changes motivated by a visually attractive frontier or by a subgroup failure. A time-stamped internal freeze is useful, but it must not be described as external preregistration unless the protocol is actually deposited in an independent registry before execution.

6.5 Common agent scaffold and execution controls

The preferred research scaffold is mini-SWE-agent, whose current documentation describes a small agent with shell-based interaction, linear conversation history and support for multiple model providers (SWE-agent Team 2026). The SWE-agent maintainers currently recommend mini-SWE-agent as the simpler successor for new use (Yang et al. 2024). Reusing this scaffold reduces implementation burden and helps make model substitution interpretable. Its published performance claims are not evidence for the proposed TTGL routing intervention.

The scaffold, prompt template, action parser, model transport, container environment and stopping policy are pinned. A compatibility suite checks each candidate model before inclusion: valid action formatting, tool-response continuation, context accounting, maximum-output handling, cancellation, malformed-response recovery and preservation of task identity. If a provider requires a different transport wrapper, that wrapper may translate protocol fields but must not add a stronger planning procedure or extra hidden context. Native reasoning and tool-calling differences are recorded as treatment characteristics.

Each task/condition/seed begins from an independent clean checkout and an identical prebuilt environment. Reuse is allowed for immutable container layers and dependency caches; writable repositories, conversation state and credentials are not shared between runs. Task order is randomized or counterbalanced across models and policies to reduce confounding from time-of-day load, provider degradation or cache warming. Concurrency limits are fixed and recorded. Model inference runs separately from the sandbox that executes repository code.

Two resource regimes are reported. The common-resource regime applies the same wall-time, agent-step and spending ceilings to all conditions, with limits selected on development tasks and frozen before test. It estimates performance under an explicit application budget. The service-profile regime uses each provider's documented recommended inference mode within a disclosed economic budget, because identical numerical token caps do not imply identical reasoning effort or cost across tokenizers. These regimes answer different questions and are not pooled into one ranking.

No model receives hidden evaluator results during its attempt. Ordinary repository tests available at the base revision may be run by every agent, and their execution cost is recorded. Final grading occurs in a fresh evaluation environment. The pinned SWE-bench documentation warns that cached evaluation results can depend on run identifier and instance identifier rather than patch content; therefore each model, policy, seed and patch receives a unique run identifier, and the evaluator record is checked against the submitted patch hash. Reusing a previous success after changing a patch is treated as a failed run-validation check.

6.6 Models, release freshness and experimental arms

Model selection follows a dated evidence review, not an unqualified claim to have tested "the latest model." A model is eligible when an exact downloadable checkpoint or stable API identifier is available, its intended use and license are understood, it supports the frozen scaffold, and it can complete development compatibility checks within the available resources. A rolling provider alias is recorded together with the resolved snapshot or response metadata; if the provider cannot expose a stable revision, that limitation is visible in the result table. New releases after the freeze are considered for a separate replication rather than silently replacing a running condition.

The current shortlist selects **Qwen3.5-4B** as the primary local candidate and **Qwen3.5-9B** as a local-capability/hardware ablation, subject to the model-card and compatibility checks recorded in the accompanying model register. **DeepSeek-V4.1-Flash-1017** is the proposed remote reference when an endpoint can be tied to that exact version; **Qwen3.8-27B** is an alternative open-weight remote control whose serving requirements must be measured. A provider alias is not assumed equivalent to the named checkpoint. These choices supersede legacy Qwen2.5 development configurations for the prospective study; the existing trainer and serving stack have not thereby been shown compatible with the newer architectures.

An advertised low active-parameter count for a mixture-of-experts model does not make its total weight footprint small. In particular, a single 80 GB training GPU is not a deployment plan for the full DeepSeek teacher. Closed-model reference systems include at least one contemporary OpenAI model and one contemporary Anthropic or Google model where authorized access and budget permit. Exact identifiers, quantization, context limits and provider prices are resolved in the model register before execution. The study does not assume that every shortlisted condition can be funded by the initial pilot budget.

Arm	Model allocation	Purpose and status
L	Local candidate for the complete agent attempt	Measures local capability and operating cost
R	One remote open-weight candidate for the complete attempt	Primary stronger-model reference; stronger performance is tested rather than assumed
D	Learned pre-call local/remote selection	Primary dual-system treatment
RAND	Random pre-call selection with frozen calibrated allocation probability	Allocation baseline, evaluated with the same local/remote pair
HEUR	Prespecified prompt-length or development-fitted simple rule	Tests whether learned routing exceeds a simple allocation rule
UNC	Router ranking predicted local failure	Separates uncertainty from predicted benefit of delegation
ESC	Bounded local work followed by registered escalation	Secondary real cascade; requires fresh continuation runs
C1/C2	Each selected closed model alone in the same scaffold	External economic/accuracy reference conditions
ORACLE	Outcome-informed selection using paired completed runs	Nondeployable diagnostic bound only
BARE	Single model with frozen context and no interactive tools	Separate interface diagnostic; not the primary routing comparator

Only L, R, D and RAND are necessary for the first software-engineering feasibility pilot. The primary confirmatory comparison is D versus R after the model pair and policy are frozen. Expanded routers and additional models establish the shape and robustness of the tradeoff but increase acquisition cost. A cheap-to-query open-weight model served by an external provider is not labelled local execution; deployment location and model openness are separate columns.

6.7 Router training and deployment policy

For task context x, let $p_L(x)$ and $p_R(x)$ estimate local and remote resolution probabilities, and let $c_L(x)$ and $c_R(x)$ estimate the expected costs of completing an attempt. The basic benefit score is $g(x) = p_R(x) - p_L(x)$. A cost-aware score compares this estimated gain with incremental cost, for example $s(x) = g(x) - \lambda[c_R(x) - c_L(x)]$. The multiplier λ expresses a declared willingness to pay for expected additional resolution probability. A ratio formulation is possible but requires care near zero or negative incremental costs; the primary implementation uses the difference formulation.

Initial predictors use transparent prompt-derived features and a regularized linear classifier, followed by a fixed pretrained embedding model as a separate candidate. Vocabulary, feature scaling, cost models, calibration and any router-model selection use development data only. A small classifier is preferable to an additional generative model unless the latter's benefit survives inclusion of its own inference cost. The chosen feature set and classifier are frozen before test. The meaning of its confidence is task-resolution probability under a particular backend and scaffold, not the model's natural-language declaration of certainty.

Paired development outcomes permit estimating benefit without assuming that the remote model always dominates. Each backend is run independently on the same development tasks and repeated seeds. A task with local success and remote failure is retained. A router trained only on failed local tasks would not observe the full deployment distribution and could overestimate escalation value; historical production traces are therefore insufficient unless their sampling policy is known and the resulting selection bias is addressed.

The primary online router uses a threshold selected on calibration data. It cannot choose the best half of an already observed test batch, because that assumes knowledge of future arrivals. Candidate thresholds are fixed on a declared grid, and one is selected using a prespecified calibration utility or quality constraint. The realized test exposure and cost are measured; they need not equal the calibration target. A fixed-budget batch-ranking analysis may be reported separately for queued workloads and explicitly identified as such.

RAND uses a frozen probability derived from the same calibration target and uses a recorded random seed. Because tasks consume unequal token counts and time, matching the number of remote calls does not ensure matching actual cost. Comparisons therefore report both remote exposure and measured cost. A supplementary matched-estimated-cost allocation experiment can use the same development-fitted cost estimates for every policy, while a post-hoc interpolation of measured curves remains descriptive. No realized test-cost threshold is tuned to favour the learned router.

6.8 Escalation and confidence features acquired during execution

The ESC treatment allows local work for a registered maximum number of steps or cost fraction, then evaluates an escalation rule. Available features may include invalid actions, unsuccessful repository tests, repeated edits, context exhaustion and progress in the visible workspace. These signals cost time and inference to acquire; their cost is charged before assessing whether escalation was worthwhile. Feature extraction must not inspect the hidden grader, reference patch or final success label.

At escalation, the system transfers a canonical issue description, current workspace diff, executed commands, observations, unresolved plan and remaining budget. Switching occurs between completed tool actions. Opaque provider reasoning state or key/value caches are not assumed transferable. A summarizer, if used, is fixed across conditions and charged to the task. The remote model’s performance on this altered state must be measured through actual continuation, because earlier local changes may help, mislead or damage the starting point.

Confidence-aware speculative decoding motivates the principle of allocating computation according to predicted useful work. It does not supply a task-level correctness guarantee. A draft-token acceptance head predicts acceptance within a particular verification algorithm; the routing model predicts issue resolution under a tool-using policy. Their targets, training labels and failure modes differ. Token log probabilities may be explored as additional signals when available, but they are not interchangeable with calibrated software-engineering success probabilities.

6.9 Outcome, router-quality and robustness measurements

The primary software-engineering metric is resolved fraction: completed candidate patches that pass the pinned issue-resolution criteria divided by all eligible assigned task attempts. Syntax errors, invalid patches, exhausted budgets and model-originated failures remain failures in the intention-to-evaluate analysis. Infrastructure failures are separately labelled and may receive one prespecified retry with identical settings when a logged independent service failure occurs. The original failure, retry cost and final disposition remain recorded; model failures are not relabelled as infrastructure problems to improve scores.

Patch validity, regression-test status, time to final patch, number of tool actions, context/token use, and user-visible failure categories are secondary measures. A blinded human review of a prespecified sample can assess patch clarity, scope and maintainability, using a rubric and inter-rater agreement. Human review is not silently substituted for benchmark resolution. Where two models produce equivalent acceptable patches but differ in style, the primary binary outcome remains the evaluator's stated criterion.

Router quality is assessed at several levels. Predictive calibration uses Brier score, log loss and reliability diagrams on unseen tasks, with binning fixed before evaluation. Discrimination metrics such as AUROC and average precision are useful diagnostics for success or beneficial-delegation labels, but a high AUROC alone does not establish a useful operating threshold. Decision quality is measured through resolved fraction at each registered cost point, regret relative to the outcome-informed oracle, and the rate of missed beneficial delegations and unnecessary expensive delegations. The oracle is computed after outcomes are collected and never used for model selection.

Coverage–risk curves describe error among tasks kept local as local coverage changes. Accuracy–cost curves describe the complete system, including delegated tasks and overhead. Both are needed: a router can make its local subset look accurate simply by delegating almost everything. A small set of predeclared cost weights or calibration operating points defines the primary frontier; dense post-hoc curves are labelled exploratory. Confidence bands or paired uncertainty estimates accompany comparisons where the sample size supports them.

Strata include repository, issue type if reproducibly annotated without outcomes, patch-size proxy available before inference, context length, local device profile and model version. True reference-patch size may be reported retrospectively as an audit characteristic but cannot be a deployment feature. Repository-level analyses are especially useful for identifying concentration of success or cost. Small strata are reported with denominators and uncertainty rather than promoted as independent discoveries.

6.10 Economic accounting and pricing comparisons

The principal economic metric is total cost per assigned task; cost per resolved task is reported alongside it as the total cost of all attempts divided by the number resolved. Failed attempts are included in the numerator. If no task is resolved, the latter quantity is undefined or unbounded and is not replaced by zero. A system

that is cheap because it produces very little useful work must not be ranked solely by average API spend.

For a provider API, task cost sums the billable categories actually reported by that provider: uncached input, cached input reads, cache creation where charged, output and reasoning tokens where separately billed, tool or search fees, retries and any batch/priority modifiers. Tokens are counted with the model's accounting conventions. Output characters are not treated as tokens, and identical nominal token counts across different tokenizers do not guarantee equal information or compute. Price quotes are dated, currency-labelled and tied to the exact provider endpoint and service tier. Promotional credits and free tiers are reported separately from ordinary prices.

For local or rented inference, cost includes occupied accelerator/CPU time, storage and transfer charges, startup and model-loading time, and the overhead of the router and sandbox. A marginal-use scenario may treat already-owned hardware as sunk capital; a fully allocated scenario includes hardware amortization, idle capacity and electricity. Both are shown when relevant, rather than describing local inference as economically costless. Energy is measured only with a documented sensor or power source; an assumed wattage produces a scenario calculation, not an empirical energy result.

The primary pre-call treatment has cost $C_{\text{router}} + (1 - a)C_L + aC_R + C_{\text{sandbox}}$, with a indicating remote selection. A local-first cascade instead includes the local work even when it later delegates: $C_{\text{router}} + C_{L,\text{spent}} + aC_{R,\text{continuation}} + C_{\text{sandbox}}$. Verification and summarization are added where they occur. Training and model distribution are allocated separately over a stated usage horizon, enabling a break-even calculation without pretending that future task volume is known.

Price-only comparisons and accuracy comparisons are distinguished. Official list prices can be compared after normalizing billable units, but they do not establish cost per resolved issue. Published leaderboard accuracies belong to particular model, scaffold, dataset version, budget and evaluation settings. They are recorded as external evidence with those fields and are not combined with our measured prices to produce a supposedly controlled ranking when the conditions differ. The strongest comparison reruns eligible open-weight and closed-model systems under the same protocol; otherwise the thesis presents an explicitly qualified landscape table.

6.11 Statistical analysis and sample-size planning

All conditions run on the same eligible tasks, enabling paired analysis. Three independently recorded decoding seeds are planned for the adequately funded confirmatory experiment. Task-level resolution probabilities are estimated from those attempts, and the primary average weights tasks equally. Seeds are repeated observations within a task, not additional independent software issues. Provider nondeterminism may persist despite requested seeds; version and timing metadata are retained.

The primary uncertainty calculation uses a paired hierarchical bootstrap that resamples repository clusters and retains matched task/condition observations within sampled clusters, with task-level repeated-seed records kept together. Repository weighting and task weighting are fixed in the analysis specification

so the reported quantity remains the task-weighted benchmark average. A task-stratified sensitivity analysis conditional on the observed repositories is also reported. The limited number of repositories means that even 500 issues may provide weak evidence for generalization to arbitrary new repositories; bootstrap intervals are not advertised as eliminating that limitation.

The confirmatory noninferiority and cost claims follow the fixed gatekeeping sequence defined above. A joint plot of paired accuracy differences and cost ratios helps interpret cases where quality and cost move in opposite directions. Secondary comparisons between learned routing, uncertainty routing, heuristics and extra models are descriptive with intervals; any formal family of significance tests uses a prespecified correction. Pareto dominance is assessed with uncertainty rather than inferred from overlapping point estimates alone.

The feasibility pilot estimates operational cost, variance and failure modes; it does not supply a final power claim. Before confirmation, sample-size simulation uses development-only estimates of local/remote discordance, repository clustering and run-to-run variability. It estimates the probability of establishing the registered margin and cost effect across plausible scenarios. If the full benchmark remains underpowered, the remedy is to collect additional independent tasks or narrow the stated conclusion, not to relax the margin after seeing test results. A failed noninferiority test is inconclusive about equivalence; a failure to detect a routing gain does not establish that routing has no value.

6.12 Student adaptation and logit-distillation extensions

The adaptation experiment starts only after a base-model reference and the unadapted router are fixed. Teacher trajectories come exclusively from development repositories and task clusters. They include observable issue context, visible commands, workspace changes and verified final patches. Hidden chain-of-thought is neither required nor presumed available. Teacher outputs are filtered through independent executable checks where feasible, and the filtering rate, failed examples and resulting task distribution are recorded.

The minimum comparison is unchanged student versus supervised fine-tuning with LoRA on verified teacher demonstrations. A token-matched same-family-teacher or additional-training control separates the source of demonstrations from the effect of extra optimization. Existing Transformers, PEFT and TRL components are preferred over a bespoke trainer. Exact weights, tokenizer, chat template, precision, rank, optimizer, learning schedule, data order and stopping rule are frozen, and three training seeds are planned for the funded adaptation study. A small pilot can check that loss and serialization work but cannot replace held-out issue evaluation.

Token-level distillation is a separate treatment. It uses a temperature-scaled KL term between teacher and student distributions only after verifying matching token identities, vocabulary coverage, sequence alignment and chat-template conventions. A shared model family is not sufficient evidence by itself. DeepSeek and Qwen tokenizers cannot be naively aligned by token index; verified-sequence distillation remains valid without that assumption. Top-k API log probabilities omit probability mass and are not labelled full-vocabulary teacher logits. If a truncated-distribution objective is used, it is defined and tested as a different approximation.

Preference learning is also distinct. DPO can consume chosen/rejected pairs labelled by executable checks, human review or an AI judge, but those label sources answer different questions. RLAIF specifically denotes AI feedback, whereas a unit-test reward is an executable verifier. A future RLAIF treatment must fix the judge, rubric, sampling scheme and reward-hacking checks. Online RL is not assumed to fit within the initial compute allocation. After any adaptation, the student is evaluated both with the frozen router and with a recalibrated router using new calibration data, separating model improvement from improved allocation. The final test remains unavailable to either training process.

6.13 Resource plan, execution stages and stopping rules

The current authorization is a single 80 GB Runpod GPU with a US$100 total Runpod ceiling for the initial training/inference experiment, including its storage, startup and shutdown costs. It does not authorize additional API-provider spending or fund an unspecified full matrix of contemporary models on 500 repository issues with three repeats, plus adaptation and external replication. External API usage requires a separately costed and authorized allocation. Container execution and all ancillary infrastructure must also appear in the prospective resource plan. Access and live prices are prerequisites to allocating compute.

The first repository-level pilot uses 12 development issues sampled by a fixed seed from six eligible development repositories where possible, two issues per repository, before outcomes are observed. It runs L, R, D and RAND once per issue, at most 48 task trajectories. The router for this pilot is trained on separate development issues; if paired training labels are not yet available, a fixed rule is substituted and the pilot is explicitly an integration study, not a learned-router comparison. Pilot tasks never enter the final confirmatory test.

Pilot spending is allocated before launch: a reserve is held for storage, output retrieval and termination; live per-task and per-hour limits determine how many complete paired blocks can be run. The intended 48 trajectories are a maximum, not a promise that they fit the cap. A paired block is started only when enough budget remains for every required condition under its registered cap. If funds are insufficient, execution stops at the last complete block and the achieved sample is reported as a feasibility sample. No outcome determines which unfinished tasks are discarded.

The pilot first validates two development tasks and the grader, then performs the bounded paired run, then stages one student-training smoke run if funds remain. Only after measured throughput, accuracy variability and total cost are available is a priced proposal made for the confirmatory matrix. A process timeout is paired with a controller deadline and verified resource termination, because ending Python does not end GPU billing. Persistent volumes and interrupted jobs are accounted for. No unrelated resource is stopped or deleted.

6.14 Reproducibility, artifacts and completion criteria

The evidence-catalog cutoff of 25 October 2026 is separate from the execution freeze. Checkpoint and runtime revisions are recorded before any paid

development run. The selected primary model pair, router features and thresholds, task splits, evaluator, harness, resource caps and model/service revision manifest are locked before the first held-out test call. Undocumented provider drift or a changed rolling alias triggers a separate versioned condition or a suspended run; outcomes from differing revisions are not silently pooled.

Each run records a task manifest, model/provider revision, agent and evaluator commits, container digests, prompts, routing policy and threshold, random seeds, raw tool/model events, submitted patch hash, usage counters, costs and grader identity. Outputs are append-only or written to unique run directories. The system refuses to reuse a cached success when the patch, environment or condition changes. Authentication values are never included in research artifacts.

The pre-results implementation specification requires four adapters around maintained components: a model endpoint adapter, a router decision hook, a cost/event recorder and a benchmark export/regrading adapter. Existing TTGL helpers can supply validation and analysis patterns, but the software-engineering runner must be tested independently. Required integration checks include exact task mapping, clean task resets, model-switch state preservation, tool-protocol errors, unavailable endpoints, budget exhaustion, cancelled runs, cache invalidation and verification of output provenance.

A benchmark run is complete only when assigned tasks have final dispositions, grader outputs correspond to current patches, every failed or skipped attempt is accounted for, costs reconcile with provider or instance records, and analysis is reproducible from frozen inputs. A model-training run is complete only when its resulting artifact is saved and hashed; a claim of improved software-engineering performance additionally requires the independent held-out evaluation. The final thesis will contain measured results only after those conditions are satisfied. Until then, the contribution is the literature synthesis, explicit experimental design and reproducible implementation plan, without implied evidence that the dual system already achieves the proposed accuracy–cost improvement.

Chapter 7

Analysis and reporting plan

7.1 Estimands and success criteria

The principal system estimand is the difference in issue-resolution probability between the dual-model condition and the declared remote-only baseline on the target task population. Its primary economic estimand is the ratio of mean total cost per assigned issue, dual divided by remote-only, under a specified deployment/accounting scenario. The paired mean cost difference is a supporting estimand. A claim of useful savings requires both acceptable quality and lower cost. A system that is cheaper because it abandons difficult issues must not be described as equivalently capable.

The primary analysis follows the gatekeeping procedure specified in the methodology: first assess the declared noninferiority condition for resolution, then assess cost superiority if that gate is passed. The router-specific comparison with a baseline matched on estimated cost using calibration data addresses a different question—whether allocation is useful—and is a prespecified exploratory contrast. Actual realized cost is reported because calibration does not guarantee an exact spending match on test tasks. An exploratory configuration that happens to look best is not substituted for the primary system after inspecting the final test.

Let $Y_{ir}^{a} \in \{0, 1\}$ indicate resolution for task i, replicate r, under condition a, and let C_{ir}^{a} be its total recorded cost. For the three-repeat confirmatory design, define task means $\overline{Y}_{i}^{a} = \frac{1}{3}\sum_{r=1}^{3} Y_{ir}^{a}$ and $\overline{C}_{i}^{a} = \frac{1}{3}\sum_{r=1}^{3} C_{ir}^{a}$. The paired task differences are $D_{i}^{Y} = \overline{Y}_{i}^{\text{dual}} - \overline{Y}_{i}^{\text{remote}}$ and $D_{i}^{C} = \overline{C}_{i}^{\text{dual}} - \overline{C}_{i}^{\text{remote}}$. The primary cost ratio divides the across-task mean of $\overline{C}_{i}^{\text{dual}}$ by the corresponding remote-only mean; it does not average per-task ratios. Assignment to all eligible tasks is the analysis denominator. Infrastructure failures, timeouts and budget exhaustion are handled by the declared failure policy, with a sensitivity analysis that preserves the initial analysis rather than selectively deleting inconvenient cases.

Repeated seeds estimate stochastic variability within a task/condition. They are not independent repositories or independent task draws. The report states whether a metric estimates a single attempt, the expected success of one sampled trajectory, or the success of a budgeted multi-attempt system. Reporting pass@k as if it were single-attempt resolution would obscure both sampling effort and cost.

7.2 Accuracy and cost frontiers

For every prespecified policy configuration, the report plots mean issue resolution against mean total cost. Markers identify model pair, routing policy, intervention point and deployment scenario. Uncertainty is attached to both coordinates or to pairwise contrasts as appropriate. Policies are considered dominated when another evaluated configuration is no less accurate and no more expensive under the same accounting assumptions, with uncertainty acknowledged. An empirical Pareto frontier is descriptive and does not establish optimality among untested policies.

Two views are required. A **fixed-quality view** asks how much cost can be reduced while remaining within the quality constraint. A **fixed-cost view** asks how much resolution improves at the same available spending. Remote-call fraction is also reported, but is not the horizontal axis for all economic conclusions: different requests can have different input/output lengths, cache behavior and reasoning budgets. A policy that sends fewer calls remotely can still cost more.

API and self-hosting scenarios are analyzed separately. The same open-weight model can be evaluated as a provider-hosted endpoint and as a self-hosted service, but the change in serving stack can alter latency, quantization and outputs. Prices from a dated catalog are not substituted for invoices when an actual call is made. Catalog-based extrapolations are labelled estimates, and intervals from task resampling do not imply certainty about future provider prices.

7.3 Router quality as a prediction problem

A routing model should be assessed independently of the end-to-end system it controls. The study therefore reports predictive discrimination, calibration and decision quality.

Discrimination measures whether higher predicted success corresponds to higher observed success. AUROC and, where class imbalance makes it useful, precision–recall summaries can describe local and remote outcome predictors. A high score does not guarantee good allocation because the router needs to identify differences between models, not merely easy tasks. Benefit ranking is assessed through matched-budget resolution, opportunity regret and gains over a random policy.

The Brier score for a binary outcome is

$$\mathrm{BS} = \frac{1}{n}\sum_{i}(\hat{p}_i - Y_i)^2.$$

When multiple samples estimate a task's success probability, the report specifies whether it scores individual Bernoulli outcomes or task means; these are different quantities. Reliability plots show bin counts and uncertainty. Expected calibration error is reported with the binning rule fixed in advance, since changes in bins can materially alter it. Proper scoring rules are primary calibration summaries; an attractive reliability plot alone is insufficient.

For an action a_i, a simple observed utility is $U_i(a_i) = Y_i(a_i) - \lambda C_i(a_i)$. An outcome-informed oracle can define an upper bound by choosing the best observed action

for each task. Regret to that oracle exposes headroom, but the bound is optimistic when it selects among noisy outcomes. It is diagnostic only, never an achievable deployed baseline or a way to choose test thresholds.

Selective-risk curves report error among tasks retained locally as a function of local coverage. Coverage must count actual local-only completion; a task using a remote planner or judge cannot be silently classified as local-only. The report also shows unresolved tasks, abstentions and remote escalations rather than counting only accepted local outputs.

7.4 Statistical uncertainty and dependence

Paired analysis uses the same tasks across conditions, reducing variation attributable to the task mix. Bootstrap or model-based uncertainty must respect the dependence structure specified by the dataset: trajectories within tasks, tasks within repositories, and potentially related issues or patches. The primary uncertainty scheme is fixed before observing test outcomes and is accompanied by the number of independent repositories/clusters.

If the available benchmark contains few repositories, a large number of tasks does not create equally many independent repository samples. Results can characterize the benchmark more precisely than they characterize arbitrary unseen repositories. Repository-level resampling, leave-one-repository-out sensitivity and the prespecified unseen-repository subset help reveal this distinction, but none makes a small set representative of all software development.

For ratios such as cost per resolved issue, uncertainty is computed from the paired numerator and denominator within each resample, not by independently combining marginal intervals. A denominator of zero produces an undefined value, which remains visible. Cost distributions are often skewed; the report includes median, upper quantiles and extreme-run attribution in addition to means.

Subgroups, multiple budgets and model pairs create multiplicity. The methodology identifies the confirmatory contrasts and correction/gatekeeping rule. Other comparisons are explicitly exploratory. The thesis will report negative and inconclusive outcomes and will avoid treating a failure to reject a null as proof of equivalence.

7.5 Power and the role of the feasibility pilot

A limited pilot serves to validate tooling, task duration, hardware memory, logging and price estimates. It is not described as a well-powered test of a small accuracy difference. The number of pilot tasks and stopping rule are frozen before outcomes are seen; a pilot task used for tuning is ineligible for a later untouched test.

A simple planning approximation illustrates the scale required for paired binary comparisons. If q is the probability that two conditions disagree and the desired detectable difference is δ, an independence approximation gives

$$n \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 q}{\delta^2}$$

for a two-sided difference test near the null. For example, **assuming** $q = 0.25$, two-sided $\alpha = 0.05$, power 0.80 and a five-percentage-point difference yields approximately 785 task pairs before accounting for clustering. This is a planning scenario, not an observed discordance rate or a claim that this sample size is sufficient for noninferiority, all policies or all repositories. The final calculation must use the declared test and a conservative dependence assumption.

The approved US$100 GPU allocation is treated as a feasibility resource. A full benchmark across several current models, multiple seeds and closed-provider APIs requires a separately costed acquisition plan. If the funded scope remains a pilot, the final thesis must describe its conclusions as preliminary and narrow its claims accordingly rather than asserting the confirmatory design was executed.

7.6 Prespecified outputs

The completed study will produce five groups of tables and figures:

1. **Cohort and execution accounting:** task/repository counts, exclusions, environment failures, seeds, model/harness revisions and actual resource use.
2. **System outcomes:** resolution, regressions, costs and latency by condition; primary paired contrasts with uncertainty.
3. **Routing diagnostics:** coverage, remote exposure, calibration, regret and prespecified subgroups.
4. **Economic scenarios:** API invoice accounting, self-hosted utilization sensitivity, local marginal versus total cost, and adaptation amortization assumptions.
5. **Student intervention:** base versus adapted model, fixed versus recalibrated router, training cost, export/quantization checks and forgetting diagnostics.

Empty result-table templates are provided in the appendix. No cells are populated from hypothetical values. Published external scores belong in the background model catalog and retain their original attribution; they do not populate the study's result tables.

7.7 Result-dependent discussion to be written later

The final discussion will answer the research questions in the order established here. If quality is retained with lower cost, it will examine the range of workloads and accounting assumptions supporting that result. If cost decreases alongside unacceptable quality loss, it will explain the tradeoff without presenting it as a successful substitute for the remote model. If differences are unresolved, it will discuss precision and sample size rather than asserting equivalence.

Possible explanations—miscalibration, inadequate features, task shift, weak local tool use, state-transfer overhead, teacher-data bias or serving incompatibility—will require diagnostic evidence. The discussion will distinguish observed mechanisms from plausible hypotheses. It will also assess whether any improvement transfers to unseen repositories, current model revisions and real local hardware. None of those conclusions is written in advance.

Chapter 8

Reproducibility, ethics and limitations

8.1 Implementation state

The research repository contains reusable components, but not all proposed empirical stages have been performed. This distinction matters because a successful import, data preparation or configuration validation is not evidence of language-model quality.

Component	Current status	Evidence it can support
Historical programming-problem data/replay pipeline	Implemented separately	Data validation and retrospective routing methodology on that distribution
Training-example preparation and LoRA scaffold	Implemented for an earlier pinned Qwen configuration	Preparation integrity and software integration; not results for the latest candidate models
Base/adapter generation scaffold	Implemented	A route to fresh inference after model/runtime validation
Tiny random-model training fixture	Software validation only	Loss masking, optimizer and adapter serialization behavior
Current-model catalog and pricing provenance	Source review	Candidate selection and conditional economic assumptions
Common-harness repository-level evaluation	Prospective	No main-study accuracy or cost result yet
Latest-model distillation and RLAIF	Prospective	No empirical improvement claim yet

In particular, an old Transformers or serving-engine pin must not be assumed to support a newly released hybrid-attention or mixture-of-experts architecture. Each selected checkpoint requires a capability preflight: tokenizer/template agreement, generation, stop behavior, output extraction, context limits, model precision and intended adapter targets. A change made to satisfy that preflight is recorded before the relevant run.

8.2 Reproducibility package

The experiment package will include immutable source references, configuration files, data split manifests, environment specifications, policy parameters and execution commands. Each released result is linked to a code revision and dependency lock. Raw artifacts are retained unchanged, and derived artifacts identify their source hashes. A rerun is written to a new directory so it cannot silently overwrite contradictory evidence.

Task selection and treatment assignment are deterministic given the recorded seed and input manifest. Where hosted providers do not guarantee deterministic sampling, repeated outcomes and provider metadata are recorded rather than asserting exact reproducibility. Reproducing a conclusion may be possible even when an identical text trajectory is not. Floating API aliases are a specific threat; the researcher should prefer snapshot IDs and record response-reported model versions when available.

The benchmark evaluator and the agent workspace are separate environments. Container image digests, operating-system packages, test commands, timeouts and resource limits are part of the evidence. Dependency failures are classified according to the prespecified rule. The researcher does not repair only the failed conditions that favor a preferred model; common environment fixes require documented reruns across affected comparisons.

Pricing records include provider, endpoint/model mapping, currency, unit, context tier, cache policy, region, batch status, retrieval date and source. Rates verified only through a community-maintained catalog remain flagged. A price table alone cannot reconstruct an invoice without usage categories. GPU records include creation/deletion timestamps, advertised rate, storage, network charges and actual billing where available.

8.3 Task management and publication

The intended project system is the Fastest Jira board, with manuscript publication in the FASTEST Confluence space. A ticket should reference an experiment stage, its acceptance criteria and immutable artifact locations. State transitions require evidence—for example, a training task is complete only after the declared run completes and its artifacts are verified, not after a script is written.

When the Atlassian connection is unavailable, local task records and import drafts are maintained with clearly marked local identifiers. Synchronization must search for existing work items before creating new ones. Publication is a distinct event from preparing a Markdown or Word document. The manuscript makes no claim that a remote page or issue exists without a confirmed service response.

8.4 Data and research ethics

Public availability is not the same as unrestricted redistribution or training permission. Benchmark tasks, repository code, model weights, software and API-generated outputs can have different licenses. The study records provenance and reviews the applicable terms for each use. A license for an inference kernel does

not establish a model-weight license; a model-weight license does not establish a hosted provider's output-training policy.

The initial study uses public benchmark repositories. If later work includes customer code, participation and training permissions must be explicit, with retention, deletion and tenant boundaries defined. Secrets, personal information and proprietary content should not be logged or added to a training set by default. Local-first operation is not a privacy guarantee if the system transmits context to a remote model; remote eligibility remains an enforceable condition of the policy.

Generated code is evaluated in isolated environments with bounded resources. The harness controls tools and credentials independently of model output, and benchmark tests are protected from modification by the agent being assessed. These controls support both reliable measurement and safe execution. They do not establish that a patch passing the benchmark is suitable for production deployment without review.

The research also considers distributional effects of the objective. Maximizing the number of solved issues per dollar may prioritize some languages, repositories or issue types. The study reports coverage and unresolved work across prespecified task strata and does not interpret those strata as a comprehensive social-fairness evaluation. A commercial policy that assigns different value to customers or tasks requires an explicitly different utility function.

8.5 Construct validity

Benchmark resolution is a narrow operational definition of software-engineering accuracy. It measures whether the final patch satisfies specified tests under a particular evaluator. It does not fully measure code readability, maintainability, security, user satisfaction or long-term regressions. A no-tools LLM, a single-model agent and a routed agent are different systems; comparisons must name the system-level intervention.

"Confidence" has several meanings: token likelihood, a model's verbal self-assessment, a classifier's predicted success probability and a policy's estimate of utility. Treating them interchangeably weakens the causal argument. The thesis defines the prediction target and evaluates calibration for that target. Speculative-decoding acceptance is not reused as a proof of task correctness.

Cost has similarly different interpretations. An API invoice, marginal electricity, rented GPU time and amortized ownership cost answer different questions. The report states which perspective supports each claim and avoids assigning a zero total cost to user-owned hardware merely because no external API call is billed.

8.6 Internal validity

Major threats include test leakage, inconsistent task environments, different prompt/tool budgets, outcome-dependent exclusions and post-hoc selection of successful models or thresholds. The protocol addresses these through immutable splits, common harnesses, blinded final-test outcomes during development, complete accounting and prespecified contrasts. Known overlaps or near-duplicates are documented rather than erased from the record after their effect becomes visible.

Published paired-output replay can support development, but it does not recreate the effect of an actual stateful escalation. It may also inherit upstream model-training contamination and unknown inference details. The prospective system experiment collects its own complete trajectories and does not infer missing latency or token usage from execution-test metadata.

Distillation adds the risk that the student is evaluated on targets or task families represented in its training data. Verification filtering and teacher-generated preferences create additional selection effects. The study holds final tests aside and compares adaptation controls, while recording which data were available to each component. A frozen router and a recalibrated router are separate conditions.

8.7 External validity

Public benchmarks represent a limited set of repositories, languages and issue types. Repeated use by model developers may make them easier than novel customer work. A temporal task split helps only if the chosen model did not see the relevant tasks during its training or post-training; a recent benchmark date alone does not establish that fact.

The study therefore combines a stable benchmark with a prespecified repository-unseen analysis and, if resources permit, an external benchmark using its own pinned harness. Scores from the external benchmark are reported separately. Failure to reproduce a ranking across benchmarks is a result about transfer, not a reason to silently choose the more favorable benchmark.

Local hardware is another source of variation. GPU memory capacity, memory bandwidth, CPU support, quantization kernels and thermal limits affect latency and practical model choice. A single 80 GB cloud GPU is useful for training and controlled inference but is not evidence about ordinary laptops. The final product claim requires separate device measurements or a clearly restricted hardware scope.

8.8 Resource and completion boundaries

The user has authorized a Runpod experiment ceiling of US$100 for one 80 GB GPU. The pilot must reserve resources for startup, model download, artifact export and verified cleanup. A timeout on a training process does not stop GPU billing. Provisioned resources are tracked and terminated through the provider, with removal confirmed. Additional provider APIs and a full multi-model study require their own resource plan; they are not silently treated as covered by that GPU budget.

The main study is complete only when its declared conditions have been executed, failures accounted for, invoices and usage reconciled, statistical analysis performed and results written without replacing the prespecified questions. If access or funding supports only a pilot, the thesis must identify that scope explicitly. Unexecuted configurations, synthetic examples and software tests cannot be promoted into empirical model-performance evidence.

Results and discussion reserved for completion

No empirical results chapter is included in this draft. After execution, it will report the complete cohort, all prespecified conditions, primary contrasts, uncertainty, calibration, cost accounting, student adaptation and failures. The discussion will interpret those observations against the research questions and limitations. The conclusion will be written only after the evidence exists.

Appendix: empty reporting templates

Main system comparison

Condition	Tasks attempted	Resolution	Total cost per issue	Cost per resolved issue	Uncertainty
Local-only agent	Pending	Pending	Pending	Pending	Pending
Open-weight remote-only agent	Pending	Pending	Pending	Pending	Pending
Closed-model remote-only agent	Pending	Pending	Pending	Pending	Pending
Dual-model policy	Pending	Pending	Pending	Pending	Pending
Random policy matched on calibration cost	Pending	Pending	Pending	Pending	Pending

Routing and student diagnostics

Quantity	Prespecified interpretation	Observation
Local coverage	Tasks completed without remote model assistance	Pending
Calibration	Predicted versus observed task/continuation success	Pending
Routing gain	Paired utility difference at comparable cost	Pending
Student effect	Base versus adapted student under the same harness	Pending
Router adaptation effect	Frozen versus recalibrated router after student update	Pending
Amortization	Training cost relative to measured eligible savings	Pending

Run manifest checklist

Required fields are task, repository and base commit; benchmark revision; harness commit; image digest; condition; model, adapter and tokenizer revisions; decoding configuration; seed where supported; routing features and decision;

context, tool, iteration and time limits; usage categories; actual billed rates; final patch; evaluator output; failure class; start and end timestamps; resource cleanup; artifact hashes; and analysis-code revision. Unknown values remain explicitly unknown.

References

Bai, Yuntao et al. 2022. ‘Constitutional AI: Harmlessness from AI Feedback’. https://arxiv.org/abs/2212.08073.

Chen, Lingjiao, Matei Zaharia, and James Zou. 2024. ‘FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance’. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=cSimKw5p6R.

Chen, Mark et al. 2021. ‘Evaluating Large Language Models Trained on Code’. https://arxiv.org/abs/2107.03374.

DeepSeek-AI. 2025. ‘DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning’. https://github.com/deepseek-ai/DeepSeek-R1/blob/main/DeepSeek_R1.pdf.

———. 2026. ‘DeepInfer: High-Performance, Large-Scale Inference Framework for DeepSeek Models’. Source code and documentation. https://github.com/deepseek-ai/DeepInfer/tree/553741557a57df44a46fb3ab72b6aafeaf945337.

Deng, Xiang, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, et al. 2025. ‘SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?’ *arXiv Preprint arXiv:2509.16941*. https://arxiv.org/abs/2509.16941.

Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. ‘QLoRA: Efficient Finetuning of Quantized LLMs’. *arXiv Preprint arXiv:2305.14314*. https://arxiv.org/abs/2305.14314.

Geifman, Yonatan, and Ran El-Yaniv. 2019. ‘SelectiveNet: A Deep Neural Network with an Integrated Reject Option’. In *Proceedings of the 36th International Conference on Machine Learning*. https://proceedings.mlr.press/v97/geifman19a.html.

Guo, Chuan, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. ‘On Calibration of Modern Neural Networks’. In *Proceedings of the 34th International Conference on Machine Learning*. https://arxiv.org/abs/1706.04599.

Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. ‘Distilling the Knowledge in a Neural Network’. https://arxiv.org/abs/1503.02531.

Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. ‘LoRA: Low-Rank Adaptation of Large Language Models’. https://arxiv.org/abs/2106.09685.

Jain, Naman, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. ‘LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code’. https://github.com/LiveCodeBench/LiveCodeBench.

Jimenez, Carlos E., John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. ‘SWE-Bench: Can Language Models Resolve Real-World GitHub Issues?’ In *International Conference on Learning*

Representations. https://openreview.net/forum?id=VTF8yNQM66.

Leviathan, Yaniv, Matan Kalman, and Yossi Matias. 2023. 'Fast Inference from Transformers via Speculative Decoding'. In *Proceedings of the 40th International Conference on Machine Learning*, 202:19274–86. https://proceedings.mlr.press/v202/leviathan23a.html.

Liu, Jiawei, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. 'Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation'. In *Advances in Neural Information Processing Systems*. https://openreview.net/forum?id=1qvx610Cu7.

Mamou, Jonathan, Oren Pereg, Daniel Korat, Moshe Berchansky, Nadav Timor, Moshe Wasserblat, and Roy Schwartz. 2024. 'Dynamic Speculation Lookahead Accelerates Speculative Decoding of Large Language Models'. *arXiv Preprint arXiv:2405.04304*. https://arxiv.org/abs/2405.04304.

Mozannar, Hussein, and David Sontag. 2020. 'Consistent Estimators for Learning to Defer to an Expert'. In *Proceedings of the 37th International Conference on Machine Learning*. https://arxiv.org/abs/2006.01862.

Ong, Isaac, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. 'RouteLLM: Learning to Route LLMs with Preference Data'. In *International Conference on Learning Representations*. https://arxiv.org/abs/2406.18665.

Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. 'Direct Preference Optimization: Your Language Model Is Secretly a Reward Model'. In *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2305.18290.

Scale AI. 2026. 'SWE-Bench Pro V2: Validated Public Split and Locked Evaluation Protocol'. Versioned benchmark documentation. https://github.com/scaleapi/SWE-bench_Pro-os/blob/66f92766bba642462d4bbe5479e83f91f9211862/v2/README.md.

Shao, Zhihong, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. 'DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models'. *arXiv Preprint arXiv:2402.03300*. https://arxiv.org/abs/2402.03300.

SWE-agent Team. 2026. 'Mini-SWE-Agent: The Minimal AI Software Engineering Agent, Version 2 Documentation'. Versioned software documentation. https://github.com/SWE-agent/mini-swe-agent/blob/04d809ceab9df28f9adaed044884180159172930/README.md.

SWE-bench Team. 2024. 'SWE-Bench Verified and Evaluation Documentation'. Official project documentation. https://github.com/SWE-bench/SWE-bench/blob/02e7a74ffd0b707aab73d203fe87bdc7c76afc8e/README.md.

Wang, Xingyao, Simon Rosenberg, Juan Michelini, Calvin Smith, Hoang Tran, Engel Nyst, Rohit Malhotra, et al. 2025. 'The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents'. https://arxiv.org/abs/2511.03690.

Yang, John, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. 2024. 'SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering'. In *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2405.15793.

Zhuo, Terry Yue, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, et al. 2024. 'BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions'. *arXiv Preprint arXiv:2406.15877*. https://arxiv.org/abs/2406.15877.